



Оглавление

Как работать с рабочей тетрадью.....	5
Анкетирование слушателя.....	6
Модуль 1 Знакомство с ОС «Альт».....	7
UNIX-подобные операционные системы.....	7
Проект GNU.....	10
ОС на основе ядра Linux.....	12
История ОС «Альт».....	13
Репозиторий Sisyphus («Сизиф»).....	15
Продукция «Базальт СПО».....	17
Модуль 2 Основы работы в командной строке.....	25
Интерфейс командной строки.....	25
Типы команд.....	29
Терминалы.....	31
Навигация по дереву каталогов.....	36
История команд.....	37
Перенаправление консольного ввода-вывода.....	38
Специальные символы командного интерпретатора.....	43
Объединение выполнения команд.....	45
Модуль 3 Получение документации.....	47
Справочная система man (UNIX).....	47
Справочная система проекта GNU – info.....	50
Дополнительные источники справочной информации в системе.....	51
Справочные ресурсы по ОС «Альт».....	52
Модуль 4 Обработка текста.....	53
Просмотр текста.....	53
Редакторы текста.....	56
Основы написания сценариев на языке командного интерпретатора.....	58
Модуль 5 Организация хранения данных.....	60
FHS – Filesystem Hierarchy Standard.....	60
Работа с дисковыми разделами.....	67
Файловые системы в ОС «Альт».....	73
Модуль 6 Управляющие конструкции командного интерпретатора.....	78
Переменные в скриптах на языке командного интерпретатора.....	78
Позиционные параметры.....	78
Управляющие конструкции условного выполнения.....	80
Конструкции циклов.....	82
Модуль 7 Файлы и их атрибуты.....	84
Типы файлов.....	84
Утилиты для работы с файлами.....	84
Индексные дескрипторы.....	86
Модуль 8 Учётные записи пользователей и групп.....	92
Пользовательские учётные записи.....	92
База паролей пользователей.....	95
Учётные записи групп.....	97
Информация по пользователям в системе.....	98
Механизмы получения административных полномочий в системе.....	99
Модуль 9 Конфигурация системы.....	102
Уровни конфигурации.....	102



Пользовательское окружение.....	102
Пользовательский профиль.....	104
Модуль 10 Разграничение доступа.....	106
Базовая модель разграничения доступа.....	106
Изменение атрибутов безопасности файлов.....	109
Дополнительные атрибуты для исполняемых файлов.....	111
Дополнительные атрибуты для каталогов.....	114
Модуль 11 Управление процессами.....	116
Атрибуты процессов.....	116
Инструменты анализа процессов в системе.....	118
Состояния процессов.....	120
Работа с заданиями (jobs).....	122
Приоритеты выполнения процессов.....	123
Модуль 12 Удаленный доступ.....	125
Сетевые настройки.....	125
Сетевые подсистемы ОС «Альт».....	126
Разрешение имен.....	129
Утилиты сетевой диагностики.....	129
SSH – Secure Shell.....	131
Модуль 13 Приложение. Сводка по командам.....	134
1. Навигация по дереву каталогов.....	134
2. Просмотр текста.....	134
3. Организация хранения данных.....	136
4. Управление файлами.....	136
5. Архивирование и сжатие данных.....	138
6. Просмотр информации об оборудовании.....	139
7. Утилиты для работы с дисковыми разделами.....	139
8. Работа с файловыми системами.....	140
9. Настройка разграничения доступа.....	140
10. Управление пользователями, паролями, группами.....	141
10.1. Утилита useradd.....	141
10.2. Утилита usermod.....	142
11. Информация о пользователях.....	142
12. Работа с процессами.....	143
13. Сетевые утилиты.....	143



Модуль 1 Знакомство с ОС «Альт»

UNIX-подобные операционные системы

Операционные системы и их компоненты

Операционная система (ОС)

В частности любая ОС на основе ядра Linux, представляет собой набор специализированных программных средств, обеспечивающих доступ пользователей к ресурсам вычислительного узла.

Ресурсы

Аппаратные средства, а так же программные сущности самой ОС

Ядро ОС

Центральная часть операционной системы, обеспечивающая приложениям координированный доступ к ресурсам. Как правило, код ядра выполняется в специальном, привилегированном режиме процессора (режиме ядра), обеспечивающим полный доступ к аппаратным средствам вычислительного узла.



Приложения ОС

Компоненты ОС, решающие дополнительные, специфические задачи. Например, редактирование текста или обработка сетевых запросов по определенному протоколу. Приложения выполняются в непривилегированном режиме, для выполнения каких-либо системных функций, например, ввод-вывод, сетевое взаимодействие, выделение памяти под хранения данных и т.п. приложения обращаются к ядру через механизм системных вызовов.

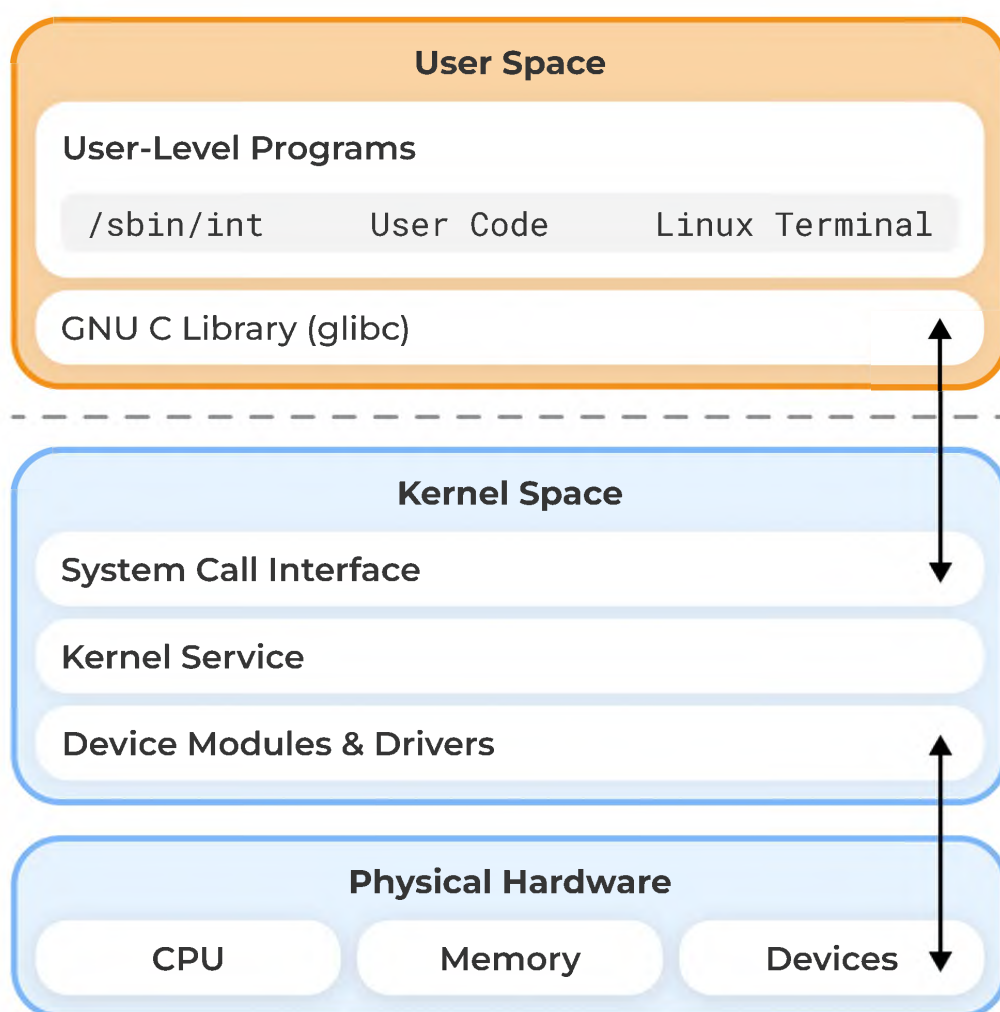


Рисунок 1: Компоненты ОС на основе ядра Linux

История появления ОС UNIX

- **ОС UNIX** – Bell Laboratories (Ken Thompson, Dennis Ritchie), 1969-1970 гг.



Универсальная операционная система с разделением времени, разработанная для компьютера DEC PDP-7 и позже, в 70е годы перенесенная на ряд других аппаратных архитектур. С 1974 года система была разрешена для использования в университетах «в учебных целях», позднее стали доступны ее коммерческие версии.

- **Язык C** – Dennis Ritchie, 1969-1973 гг.

Архитектурно-независимый, но при этом достаточно эффективный язык общего назначения. В 1973 г. ядро ОС UNIX было переписано на языке C, что упростило ее портирование на другие аппаратные архитектуры. Таким образом, возникло большое количество UNIX-подобных ОС, имеющих свои различия, но при этом обладающими общими особенностями.

Особенности UNIX-подобных ОС

1. Переносимость с платформы на платформу (язык C):

Unix-подобные операционные системы писались на языке C – достаточно низкоуровневом для обеспечения эффективности системного кода, но при этом независимом от процессорной архитектуры, т.е. ОС достаточно легко становились переносимыми.

2. Переносимость приложений (стандарты – POSIX, SUS, LSB):

Для обеспечения совместимости приложений между различными ветками UNIX-подобных операционных систем были придуманы стандарты, описывающие как пользовательские интерфейсы (например POSIX.2 – интерфейс командной строки), так и интерфейсы программирования, например, стандартизация системных вызовов.

3. Универсальный интерфейс между приложениями (текст):

Типовым интерфейсом между приложениями для UNIX-подобных операционных систем является текст и используется принцип его построчной обработки. Что дает возможность создавать достаточно сложные командные конструкции, выполняющие как по конвейеру обработку данных для получения желаемого результата.

4. Единый пользовательский интерфейс для интерактивной работы и автоматизации (Shell):



Для интерактивной работы пользователя/администратора и для создания решений автоматизации применяется единая среда (командный интерпретатор). Как следствие, навыки каждодневной работы с системой легко конвертируются в навыки создания инструментов автоматизации.

5. Удачная система разграничения привилегий (всё – файл):

Всё, или почти всё в системе – файл, т.е. задачи разграничения доступа легко сводятся к задачам уровня – может ли этот пользователь таким методом доступа взаимодействовать с таким-то файлом.

6. Многозадачность, многопользовательский доступ:

Выполнение полезной нагрузки системы в виде изолированных процессов, позже, как вариант потоков. Поддержка множества пользовательских сеансов как локальных, так и удалённых.

7. Терминальный и сетевой доступ:

Поддержка как локального, так и удалённого графического и консольного доступа пользователей к системе.

Проект GNU

Открытое и свободное программное обеспечение

- Открытое программное обеспечение (**OSS** – Open-Source Software)

ПО, исходный код которого доступен для просмотра, изучения и изменения, что позволяет убедиться в отсутствии уязвимостей и неприемлемых для пользователя функций, принять участие в доработке самой открытой программы, использовать код для создания новых программ и исправления в них ошибок – через заимствование исходного кода, если это позволяет совместимость лицензий, или через изучение использованных алгоритмов, структур данных, технологий, методик и интерфейсов.

- Свободное программное обеспечение (**Free Software**)

ПО, пользователи которого имеют права («свободы») на его неограниченную установку, запуск, свободное использование, изучение, распространение и изменение (совершенствование), а



также распространение копий и результатов изменения. СПО является подмножеством открытого ПО.

Проект GNU

- **GNU** – рекурсивный акроним GNU's Not UNIX (1983 г, Richard Matthew Stallman)

Свободная ОС (т.е. ОС, распространяемая под свободной лицензией), разрабатываемая с 1983 г. Richard Matthew Stallman, а далее в рамках **Проекта GNU**

- Проект GNU – <https://gnu.org>

Проект разработки свободной UNIX-подобной операционной системы, лишенной недостатков закрытых UNIX-подобных систем. По задумке не должен был содержать кода UNIX, при этом реализовывать все особенности UNIX-подобных ОС. В рамках проекта разработано достаточно большое количество системного и пользовательского ПО, от базовых утилит и инструментов разработки до пользовательских приложений с ГПИ.

- GNU Hurd

Официальное ядро операционной системы GNU. Серьезного распространения не получило.

Свободные лицензии проекта GNU

- **GNU GPL** – GNU General Public License – Универсальная общественная лицензия GNU – актуальная версия 3.0

Лицензия на свободное программное обеспечение, созданная в рамках проекта GNU в 1988г., по которой автор передает программное обеспечение в общественную собственность. Цель GNU GPL – предоставить пользователю права копировать, модифицировать и распространять (в том числе на коммерческой основе) программы, а также гарантировать, что и пользователи всех производных программ получают вышеперечисленные права.

- Copyleft

Принцип «наследования» прав всех производных программ, придуманный Ричардом Столлманом. Данный принцип в



лицензии GNU GPL не позволяет включать пролицензированную программу в проприетарное (несвободное) ПО.

- GNU Lesser General Public License (LGPL)

Лицензия на свободное программное обеспечение, используется, как правило, для лицензирования разделяемых библиотек. Позволяет разработчикам использовать и внедрять ПО под этой лицензией, в их собственном (даже проприетарном) ПО, без необходимости предоставления исходного кода собственных компонентов под копилефт лицензией (копилефт только для самой библиотек).

ОС на основе ядра Linux

Проект Linux

- **Linux** – <https://kernel.org/>

Ядро ОС, разработанное финским студентом Линусом Торвальдсом в 1991 и выпущенное под лицензией GPL.

- **GNU/Linux**

Выпуск ядра Linux под лицензией GNU GPL позволил его использовать вместе с наработками проекта GNU и получился, таким образом, полный стек (ядро ОС + системные и прикладные компоненты ПО).

ОС и дистрибутивы

- ОС на основе ядра Linux

ОС, в которой ядерная часть реализована на основе проекта Linux. Фактически современные ОС, на основе ядра Linux как отечественные, так и иностранные, представляют собой сочетание ядра Linux + программное обеспечение проекта GNU, плюс программное обеспечение разработанное в рамках других открытых проектов. Плюс (возможно, но необязательно) собственные дополнительные программные компоненты, разработанные производителем дистрибутива. В ряде случаев эти собственные разработки ведутся под открытыми лицензиями («Базальт СПО»), в ряде случаев – под проприетарными.

- Дистрибутив



Форма распространения программного обеспечения.
Дистрибутив Linux – форма распространения ОС на основе ядра Linux, содержащая определенный, возможно специфический набор внеядерных компонент и механизм установки ОС на вычислительный узел.

История ОС «Альт»

UNIX-наследие в ОС «Альт»

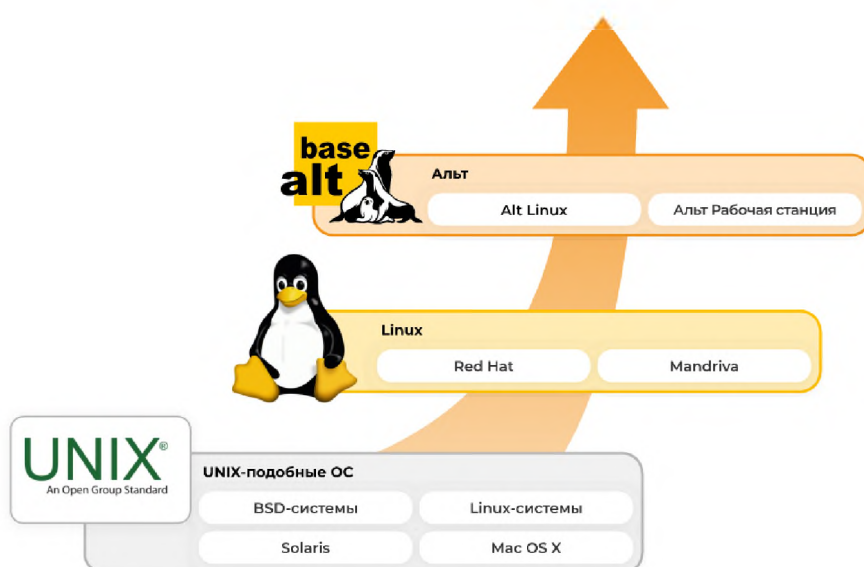


Рисунок 2: UNIX-наследие в ОС «Альт»

От ALT Linux к «Базальт СПО»

- По материалам
 - <https://www.altlinux.org/Факты>
 - <https://www.altlinux.org/History/FAQ>
 - <http://rus-linux.net/kos.php?name=/papers/history/lh-05.html#alt>
- **2001г.** Объединение команд
 - IPLabs linux Team
 - LRN (linux.ru.net)
- Основание компании **ALT Linux**



- Отцами-основателями компании были Стас Иевлев, Дмитрий Левин, Алексей Новодворский, Алексей Смирнов, Антон Фарыгин

Работа проводилась на созданной пакетной базе, наработанной еще в IPLabs linux Team, но поначалу по договоренности с компанией Mandrakesoft использовали их бренд дистрибутива – Mandrake

- Linux Mandrake Russian Edition Spring 2001 (25 Марта 2001 г.)

Дистрибутив вышел под брендом Mandrake, использовался инсталлятор от Mandrake и программы настройки от него, но пакетная база полностью поддерживалась ALT Linux. Эта пакетная база позднее станет репозиторием Sisyphus.

- ALT Linux MSI (2001, июнь)

Первый дистрибутив под собственной маркой.

- ALT Linux Junior 1.1 (2001, сентябрь)

Переход на использование связки apt+rpm для управления пакетами.

- Собственный инсталлятор и конфигуратор (2005-2006 гг)

Вместо ранее используемых Mandrake-овских. Переход на версии 3.0 дистрибутивов.

- «Базальт СПО» (сентябрь 2015 г. — настоящее время)

Занимается разработкой, продажей и поддержкой дистрибутивов под торговой маркой «Альт», разработкой инфраструктуры и пакетной базы Sisyphus.

- Выход «Альт Платформа» (2024 г.)

Технологический комплекс для сборки общесистемного и прикладного программного обеспечения и выпуска дистрибутивов. В состав комплекса входят ALT Platform Builder, Gear — инструмент для создания пакетов RPM из git-репозитория, Hasher — инструмент для воспроизводимой сборки RPM-пакетов, Alterator-mirror — служба для создания локальных копий репозитория и Mkitage-profiles — набор готовых профилей для сборки дистрибутивных образов



Репозиторий Sisyphus («Сизиф»)

О репозитории

- Репозиторий свободных программ «Сизиф» (Sisyphus)

Ежедневно обновляемый банк программ (репозиторий пакетов). На его основе создаются все дистрибутивы «Альт». «Сизиф» поддерживается «Базальт СПО» и командой разработчиков ALT Linux Team.

- Целостность с точки зрения зависимостей

В любой момент репозиторий является целостным, то есть в нём разрешены все зависимости пакетов. Это даёт возможность получать из доступного по сети репозитория программные пакеты со всеми необходимыми для их работы зависимостями и таким образом обеспечивается работоспособность устанавливаемого из репозитория ПО.

- Один из крупнейших репозиториях в мире согласно ресурсу Repology

В отличие от аналогов целиком и полностью находится в отечественной юрисдикции. Особое внимание уделяется защите системы, интернационализации, полноте и корректности зависимостей.

- **Открытая** модель разработки

Разработка «Сизиф» полностью открыта. Все инструменты, используемые для подготовки и сборки пакетов, контроля зависимостей, тестирования работоспособности, а также для подготовки конечных решений (дистрибутивов) доступны в том числе и сторонним разработчикам.

- <https://packages.altlinux.org>

Сведения о пакетной базе, текущее содержимое репозитория.

Жизненный цикл продуктов на основе репозитория Sisyphus

- Нестабильная ветка

Ежедневно изменяющийся репозиторий содержит самое новое программное обеспечение, со всеми его преимуществами и



недостатками (иногда ещё не известными). Может использоваться квалифицированными разработчиками и системными администраторами.

- Стабильные ветки

Путем фиксации состояния репозитория происходит его стабилизация, исправление проблем работоспособности, зависимостей, разного рода недочетов. Таким образом формируются стабильные ветки репозитория, подходящие для использования в ежедневной работе корпоративными пользователями. Обеспечивается стабильность и предсказуемость работы приложений. По ПО в стабильной ветке оказывается техническая поддержка.



Рисунок 3: Создание продуктов на основе Sisyphus

На базе репозитория «Сизиф» периодически формируется стабильная ветка (программная платформа), которая поддерживается в течение длительного времени и используется в качестве базы для построения очередного поколения дистрибутивов линейки «Альт».

- Текущая стабильная ветка: **P11**

Каждая стабильная ветка (branch) разработки имеет APT-репозиторий. Поскольку стабильные ветки достаточно консервативны по изменениям, то эти репозитории достаточно безопасны для использования вместе с дистрибутивами.

- **Дистрибутив ОС «Альт»** (как правило распространяется в виде установочного ISO-образа)

Дистрибутивы делаются на базе стабильной ветки (платформы). Дистрибутив отличается от репозитория тем, что в него входят только нужные программные продукты (не всё что есть в репозитории). Представляет собой интегрированную рабочую среду, предназначенную для решения тех или иных задач



пользователей. Дистрибутивы операционных систем «Альт» отличаются друг от друга кругом решаемых задач, комплектацией программного обеспечения, дизайном.

- Версия основных продуктов: **11**

Установленные из образов дистрибутивы ОС «Альт» настроены на сетевой APT-репозиторий соответствующей ветки и таким образом в ОС может быть установлено ПО, содержащееся в репозитории, и выполнено обновление существующего.

Продукция «Базальт СПО»

Доступность продуктов

- Доступность продуктов
 - <https://getalt.ru>
 - <https://basealt.ru>
- **Бесплатное использование** физическими лицами (кроме «Альт СП»)

Все дистрибутивы, кроме сертифицированного доступны для бесплатного использования физическими лицами

- Бесплатное использование в целях тестирования

Юридические лица могут использовать дистрибутивы бесплатно только в целях тестирования, в противном случае требуется покупка лицензий.

ОС «Альт Рабочая станция»

Основное решение «Базальт СПО» для **корпоративных рабочих мест**.

Дистрибутив, включающий в себя операционную систему и набор приложений для полноценной работы, поддерживающий различное дополнительное оборудование. Дистрибутив рекомендуется для обычных корпоративных рабочих станций.

Исполнена в двух вариантах — среда рабочего стола (графическая оболочка) **Gnome*** и среда рабочего стола **KDE (ОС «Альт Рабочая станция К»)**



Модуль 1

17

Знакомство с ОС «Альт»

Среды рабочего стола включают набор приложений для полноценной работы. Интегрируется как рабочее место в инфраструктурные решения, обеспечиваемые серверными дистрибутивами.

** до версии 11 применялось окружение рабочего стола MATE*

Входит в единый реестр российских программ (№ 1292)

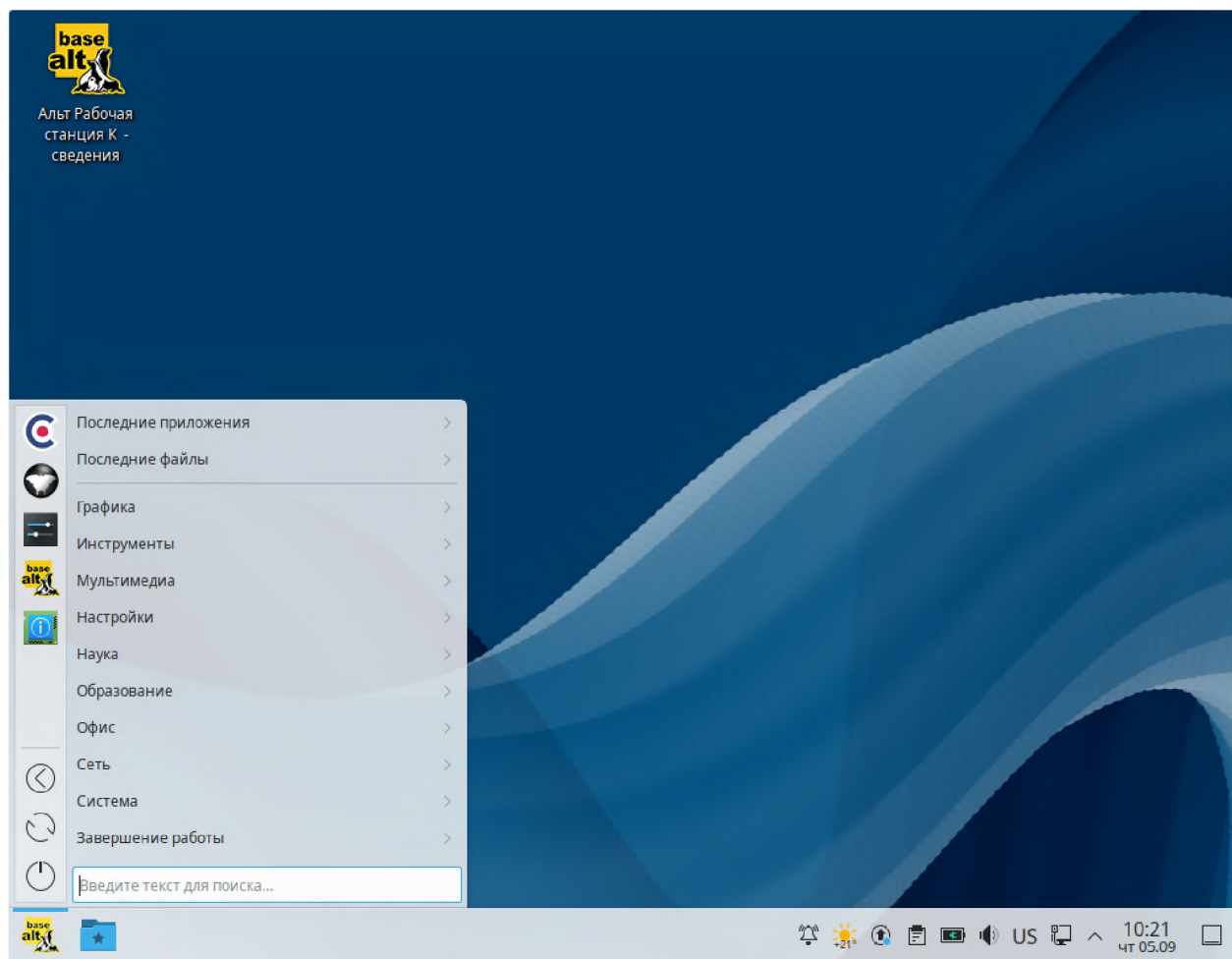


Рисунок 4: Внешний вид «Альт Рабочая станция K»

ОС «Альт Сервер»

Многофункциональное решение на основе ядра Linux для серверов с широкой функциональностью, позволяющий



поддерживать корпоративную инфраструктуру, а также различное дополнительное оборудование, прежде всего, предназначен для использования в корпоративных сетях.

- Единый реестр российских программ (№ 1541)

Предоставляет комплекс серверных приложений, оснащённый удобным пользовательским интерфейсом для настройки.

ОС «Альт Виртуализация»

- «Альт Виртуализация» («Альт Сервер Виртуализации»)

Многофункциональный дистрибутив для серверов, прежде всего, предназначен для создания виртуальной среды, обеспечивающей функционирование виртуальных машин и управление ими.

- 4 основных типа установки:
 - базовый гипервизор KVM;
 - кластер виртуализации PVE (для x86_64 и aarch64);
 - облачная виртуализация OpenNebula;
 - контейнерная виртуализация (LXC/LXD, Docker/Podman).

ОС «Альт Образование»

- Альт Образование

Дистрибутив, ориентированный на повседневное использование при планировании, организации и проведении учебного процесса в образовательных учреждениях (от дошкольных до ВУЗов).

- Единый реестр российских программ (№ 1912)

Дистрибутив содержит очень большой выбор предварительно установленных образовательных программ. «Альт Образование» – универсальный дистрибутив, он включает в себя рабочую станцию и серверные компоненты.

- Две графические среды

«Альт Образование» выпускается с графической средой XFCE и возможностью установки «из коробки» графической среды KDE5.



Модуль 1

19

Знакомство с ОС «Альт»

ОС «Simply Linux»

- Simply Linux (Симпли Линукс)

Бесплатная (без ограничений) операционная система для персональных компьютеров.

- Легковесная графическая среда XFCE4

Simply Linux – адаптированная для обычного пользователя операционная система. Под «обычным пользователем» понимается человек, не владеющий тонкостями настройки и использования системы.

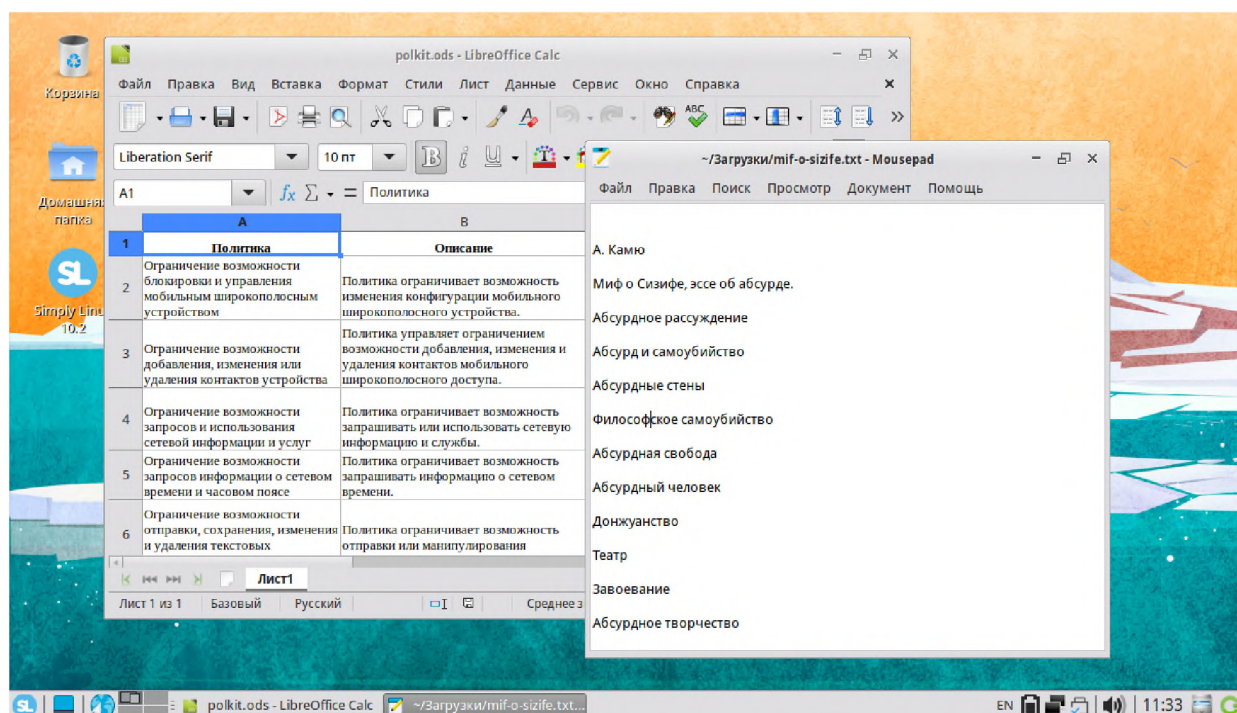


Рисунок 5: Внешний вид дистрибутива Simply Linux

ОС «Альт СП» («Альт 8 СП»)

Сертифицированный дистрибутив

- «Альт СП» («Альт 8 СП»)



Дистрибутив операционной системы для серверов и рабочих станций со встроенными программными средствами защиты информации, сертифицированный ФСТЭК России.

- 2 варианта исполнения
 - Альт СП Сервер
 - Альт СП Рабочая станция
- **Сертификат ФСТЭК России** – требования к ОС 4го класса защиты, профиль ИТ.ОС.А4.ПЗ
 - для применения в государственных информационных системах 1 класса защищенности;
 - для применения в автоматизированных системах управления производственными и технологическими процессами 1 класса защищенности;
 - для применения в информационных системах персональных данных при необходимости обеспечения 1 уровня защищенности персональных данных;
 - для применения в информационных системах общего пользования II класса.

Особенности ОС «Альт»

- Выпускается для 10 аппаратных платформ (i586, x86_64, aarch64, armh, mipsel, «Эльбрус» (e2kv3/v4/v5/v6), ppc64le, RISC-V)

А также ведется портирование на другие платформы.

- Единый графический и веб-интерфейс для настройки (alterator).
Более 100 модулей



Модуль 1

21

Знакомство с ОС «Альт»

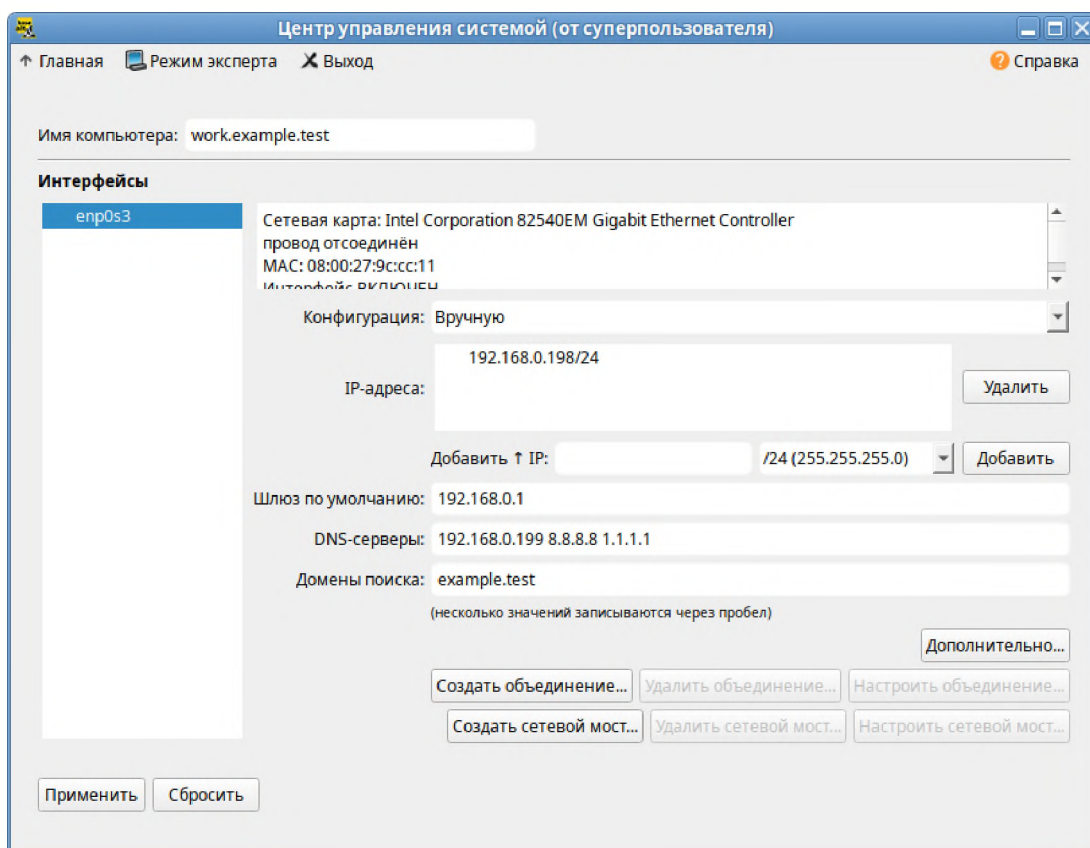


Рисунок 6: Модуль управления сетевыми интерфейсами в GUI alterator



Имя компьютера: host-201

Интерфейсы: enp0s3

Сетевая карта: Intel Corporation 82540EM Gigabit Ethernet Controller
провод подсоединён
MAC: 08:00:27:5c:d8:30

Версия протокола IP: IPv4 ☒ Включить

Конфигурация: Вручную

IP-адреса: 192.168.0.150/24 Удалить

Добавить IP: /24 (255.255.255.0) Добавить

Шлюз по умолчанию: 192.168.0.1

DNS-серверы: 8.8.8.123 8.8.8.8

Домены поиска:
(несколько значений записываются через пробел)

Дополнительно...

Создать объединение... Удалить объединение... Настроить объединение...

Создать сетевой мост... Удалить сетевой мост... Настроить сетевой мост...

Применить Сбросить

Рисунок 7: Модуль управления сетевыми интерфейсами в WUI alterator

Позволяет настраивать с графического интерфейса и удаленно (через Web) различные параметры системы, решая таким образом наиболее типовые задачи системного администратора без необходимости работать в командной строке.

- Совместимость с программным и аппаратным обеспечением Российских производителей
<https://www.basealt.ru/products/informacija-o-sovmestimosti>

Пополняемая таблица совместимости для последних версий продуктов

- Использование наработок проекта **OpenWall Linux**

TCB, pam_passwdc

- Подсистема конфигурирования сети **etcnet**

Написана разработчиками дистрибутива на основе **iproute2**



- Управление программным обеспечением средствами **apt-get** поверх **rpm**

Взяты наработки проектов RedHat Linux, Debian GNU/Linux, Connectiva

- Широкий выбор графических сред

Как среды по умолчанию в различных продуктах, так и возможность доустановки из пакетной базы

- Полный набор инструментов разработчика

Широкий выбор средств разработки и инструментальных средств для создания приложений. Как для Sisyphus, так и для собственных проектов



Модуль 2 Основы работы в командной строке

Интерфейс командной строки

Основные типы пользовательского интерфейса

- ТПИ – текстовой пользовательский интерфейс (Text user interface, TUI; также Character User Interface, **CUI**)

Разновидность интерфейса пользователя, использующая при вводе-выводе и представлении информации исключительно набор буквенно-цифровых символов и символов псевдографики.

- Интерфейс командной строки (Command line interface, **CLI**)

*Частный случай ТПИ. Способ взаимодействия между человеком и компьютером путём отправки компьютеру команд, представляющих собой последовательность символов. Команды интерпретируются с помощью специального интерпретатора – **командного интерпретатора**.*

- Графический интерфейс пользователя (ГИП), графический пользовательский интерфейс (ГПИ) (Graphical User Interface, **GUI**)

Система взаимодействия с пользователем, основанная на представлении всех доступных пользователю системных объектов и функций в виде графических компонентов экрана (окон, значков, меню, кнопок, списков и т. п.).

Командный интерпретатор

- **Командный интерпретатор**, интерпретатор командной строки

Компьютерная программа, часть операционной системы, обеспечивающая базовые возможности управления компьютером посредством интерактивного ввода команд через интерфейс командной строки или последовательного исполнения пакетных командных файлов.

- DOS: COMMAND.COM
- Windows: cmd.exe, PowerShell
- Unix: sh, bash, csh (C shell), ksh (Korn shell), zsh (Z shell)



- **Интерактивный** режим работы

Возможность ввода команд интерактивно, взаимодействуя с интерфейсом командной строки.

- Приглашение ввода команд

Заканчивается обычно символами \$ или #. Говорит о том, что командный интерпретатор готов к обработке пользовательских команд (работает в интерактивном режиме).

```
[sysadmin@altwks:/etc]$
```

Приглашение по умолчанию в ОС «Альт» состоит из: имя пользователя, имя узла, текущий каталог. Может переопределяться через переменные окружения – переменная PS1.

- **Пакетный** (неинтерактивный) режим работы

Поддерживает написание и выполнение скриптов – для автоматизации однотипных/периодических задач.

Роль командного интерпретатора в системе

1. Формирует интерфейс командной строки

Позволяет пользователю запускать другие программы и, тем самым, решать свои задачи в рамках ОС.

2. Запускается при входе пользователя в систему

Является первым процессом, запускаемым при входе пользователя в систему (при входе посредством алфавитно-цифрового терминала).

3. Формирует пользовательское окружение

Командный интерпретатор формирует пользовательское окружение – среду для выполнения команд пользователя (переменные, псевдонимы).

4. Выполняет скрипты (сценарии)

Позволяет использовать знакомый командный синтаксис интерактивного режима для создания решений по автоматизации задач администрирования.



Разновидности командных интерпретаторов в ОС «Альт»

- **sh** – Bourne Shell

*Классический командный интерпретатор UNIX-систем, предоставляет основные возможности интерактивной работы, как правило используется в пакетном режиме, т.к. поддерживается практически повсеместно. Скрипты, написанные для обработки средствами **sh** являются максимально переносимыми между системами.*

- **bash** – Bourne again shell

Усовершенствованная и модернизированная вариация командной оболочки Bourne shell. Одна из наиболее популярных современных разновидностей командной оболочки. Как правило, является командным интерпретатором по умолчанию для пользователя в ОС Linux.

- **cs**h – C shell

Командная оболочка UNIX со встроенным скриптовым языком, разработанная Биллом Джоем, активным разработчиком BSD UNIX и создателем редактора vi, в 1979 году. Базировался на коде командного интерпретатора шестой версии UNIX. Скриптовый язык не уступает Bourne shell по мощности, но отличается синтаксисом.

- **ksh** – Korn shell

Командная оболочка UNIX, разработана Дэвидом Корном (AT&T) в 1980-х годах. Имеет полную обратную совместимость с Bourne shell и включает в себя возможности C shell.

- **zsh** – Z shell

Одна из современных командных оболочек UNIX, использующаяся непосредственно как интерактивная оболочка, либо как скриптовый интерпретатор. Zsh является расширенным bourne shell с большим количеством улучшений.

Стандартный синтаксис POSIX.2

- Стандартный синтаксис командной строки



Модуль 2

27

Основы работы в командной строке

Определен во второй части стандарта POSIX, поддерживается всеми командными интерпретаторами

```
$ ls -l -rt --color=always -I "*.mp3" /tmp /srv
```

Пример командной конструкции, соответствующей стандарту POSIX.2

```
$
```

Приглашение к вводу команд

```
ls
```

Пример команды

```
-l
```

Одиночная односимвольная опция. Принято начинать с одного дефиса. Длинный вывод (атрибуты файлов).

```
-rt
```

Две односимвольные опции, состыкованы. Сортировка по времени изменения и обратная (reverse) сортировка.

```
--color=always
```

Многосимвольная опция. Принято начинать с двух дефисов

```
-I "*.mp3"
```

Игнорировать файлы в соответствии с заданным шаблоном

```
/tmp /srv
```

Параметры – объекты, в отношении которых будет выполняться команда

- Разделители команд, опций, параметров



Пробельные символы. Один либо несколько пробелов.

- **LF** – символ завершения командной конструкции

Перенос строки. Формируется драйвером терминала при нажатии клавиши **Enter**. Получение символа переноса строки говорит командному интерпретатору о необходимости начать выполнять введенную командную конструкцию.

Дополнение имён в командном интерпретаторе

- Клавиша **Tab**

Использование клавиши **Tab** позволяет упростить ввод команд.

- Дополняются:
 - имена команд/функций/псевдонимов
 - имена файлов
 - имена переменных
 - опции/аргументы

Если однозначно предполагаемое для дополнения имя не определить, **Tab** не выводит ничего.

- Повторное нажатие **Tab**

Повторное нажатие **Tab** выводит возможные варианты (если есть).

Типы команд

- Типы команд
 - внутренние команды командного интерпретатора
 - внешние команды
 - синонимы команд

Определение типа команды

- Команда **type**

Позволяет определить типы команд.

```
$ type pwd
pwd is a shell builtin
$ type ls
ls is aliased to `ls --color=auto'
```



```
$ type find
find is /usr/bin/find
```

Внутренние команды командного интерпретатора

Внутренние команды реализованы в самом командном интерпретаторе

```
$ type pwd
pwd is a shell builtin
$ type cd
cd is a shell builtin
$ type echo
echo is a shell builtin
$ type -a echo
echo is a shell builtin
echo is /bin/echo
```

Внешние команды

Реализованы в виде отдельных исполняемых файлов.

- Команда **which**

Расположение исполняемого файла команды в системе:

```
$ type cal
cal is /usr/bin/cal
$ type find
find is /usr/bin/find
$ which bash
/bin/bash
```

Синонимы (псевдонимы) команд

Обычно короткие, более удобные имена длинным командам:

```
$ alias
alias grep='grep --color=auto'
alias ls='ls --color=auto'
$ type -a ls
ls is aliased to `ls --color=auto'
ls is /bin/ls
```

Задание синонимов

```
$ alias mycal="cal 2020"
```



```
$ mycal
```

Обычно в файлах инициализации командного интерпретатора для пользовательских настроек интерпретатора (.bashrc).

Терминалы

Назначение терминалов

- **терминалы**

Устройства, предназначенные для ввода и вывода данных и организации таким образом человеко-машинного интерфейса.

- **алфавитно-цифровые** терминалы

Позволяют реализовать TUI и CLI.

- **графические** терминалы

Позволяют реализовать GUI, а так же TUI и CLI за счет приложений-эмуляторов.

- **ввод и вывод данных**

Терминал состоит из устройств вывода – дисплей (ранее м.б. принтер) и ввода – клавиатура, мышь и т.п.

- **консоль**

Термин, который часто используется вместо термина терминал, либо как синоним для алфавитно-цифрового терминала. Сочетание устройств ввода-вывода (видеоадаптер, монитор и клавиатура), используемое для организации взаимодействия с пользователем.

Организация командного интерфейса

- Двусторонний попеременный диалог между пользователем и операционной системой (процессами)

Процесс ввода команд пользователем посредством клавиатуры и получения результата их выполнения на дисплее терминала.



Рисунок 8: Двусторонний попеременный диалог

- Соответствие процессов и терминалов

Процессы, предусматривающие взаимодействие с пользователем посредством TUI и CLI, имеют привязку к терминалу, т.н. управляющий терминал, посредством которого и взаимодействуют с пользователем.

```
$ ps
  PID TTY          TIME CMD
  7309 pts/1        00:00:00 bash
 10521 pts/1        00:00:00 ps
```

Список процессов – командный интерпретатор и команда `ps`, сопоставленные текущему терминалу – `pts/1`.

- Драйвер терминала (tty)

Программное обеспечение, обеспечивающее передачу команд от устройств ввода пользовательскому процессу, и результатов выполнения команд от пользовательского процесса на устройства вывода.

- Управление драйвером терминала

Производится путем обработки специальных управляющих символов и ESC-последовательностей. Драйвер, например, может обеспечивать возможность произвольного позиционирования курсора или изменять цвет отображения символов.

Виды терминалов

- Виды терминалов с точки зрения реализации
 - аппаратные



- виртуальные
- псевдотерминалы
- **Аппаратные** или физические терминалы

Специальные аппаратные устройства, включающие дисплей (как правило алфавитно-цифровой), клавиатуру, последовательный порт для взаимодействия с ЭВМ и специальное программное обеспечение для обработки управляющих последовательностей символов. В настоящий момент большая редкость.

- **Виртуальные** терминалы

Терминалы, эмулируемые специальным драйвером (драйвером виртуальных терминалов) для организации взаимодействия с пользователем через сочетание консольных устройств.

Ctrl+Alt+F1 (2-6)

Переключение на виртуальные терминалы. При установке компонент графического интерфейса в ОС «Альт», первый виртуальный терминал используется как графический, остальные со 2 по 6 – как алфавитно-цифровые

Ctrl+Alt+F12

Переключение на системную консоль (системный виртуальный терминал), на которую ОС выводит свои сообщения в процессе загрузки и работы системы.

- **Псевдотерминалы**

Терминалы, эмулируемые специальной программой-посредником (программным эмулятором терминала) для организации взаимодействия с пользователем

- Графические эмуляторы терминала (xterm, mate-terminal, konsole, и т.д.)

Приложения, эмулирующие алфавитно-цифровой терминал поверх графического для возможности работы с интерфейсом командной строки в графической среде.

- Эмуляторы терминала при удаленном терминальном доступе



Аналогично при подключении пользователя посредством протокола удаленного терминального доступа (например, telnet, ssh), соответствующее приложение клиент выполняет эмуляцию терминала.

Режимы ввода-вывода для алфавитно-цифрового терминала

- **Канонический режим** (canonical или cooked)

Ввод данных происходит построчно, завершается символом перевода строки. До ввода завершающего символа есть возможность редактировать строку. Канонический режим используется для организации командного интерфейса и простейших TUI.

- **Неканонический режим** (non-canonical или raw)

Данные не группируются в строки, а передаются группами символов. Используется полноэкранными утилитами (vim, less, а также построенными на библиотеке ncurses – mc и т.п.). В неканоническом режиме терминал рассматривается как матрица символов, которыми можно произвольно управлять.

Управляющие символы

- Управление драйвером терминала при **вводе**

Пользователь, взаимодействуя с терминалом, вводит специальные символы, которые служат командами для драйвера терминала. Тем самым реализуется управление курсором в командной строке, либо посылка сигналов процессу, обрабатывающему ввод с терминала.

Последовательность	Объект	Действие
Ctrl-c	Процесс	Завершение процесса
Ctrl-z	Процесс	Приостановка процесса
Ctrl-\	Процесс	Прерывание процесса с дампом памяти
Ctrl-s	Процесс	Приостановить вывод
Ctrl-q	Процесс	Продолжить вывод
Ctrl-d	Процесс	Конец потока данных
Ctrl-l	Терминал	Очистка экрана



Последовательность	Объект	Действие
Ctrl-_	Терминал	Отмена выполненного редактирования
Ctrl-v	Терминал	Не интерпретировать следующий специальный символ
Enter	Строка	Конец строки
Ctrl-a	Строка	Перевести курсор на начало строки
Ctrl-e	Строка	Перевести курсор в конец строки
Alt-b	Строка	Перевести курсор на слово назад
Alt-f	Строка	Перевести курсор на слово вперед
Ctrl-w	Строка	Вырезать слово слева от курсора
Ctrl-y	Строка	Извлечь из буфера
Ctrl-k	Строка	Удаление символов до конца строки
Ctrl-u	Строка	Удаление символов до начала строки



Модуль 2

35

Основы работы в командной строке

- **stty**

Команда, позволяющая управлять режимами работы терминала, в том числе и обработкой управляющих символов.

```
$ stty -a
```

Отображение текущих настроек работы терминала.

```
$ reset
```

Переинициализация терминала.

Навигация по дереву каталогов

Команда **ls**

```
$ ls
```

Вывод информации по файлам и каталогам.

- См Приложение. п.1.1

Команда **cd**

```
$ cd [dir]
```

Смена текущего каталога.

- См Приложение. п.1.2

Команда **pwd**

```
$ pwd
```

Отображение текущего каталога.

Файловый менеджер **mc**

```
$ mc
```

Двухпанельный менеджер файлов.



Абсолютные и относительные путевые имена

- Корень иерархии каталогов

Наклонная черта – самый первый (верхний) каталог в иерархии.

- Абсолютное путевое имя файла

Абсолютный путь. Начинается с наклонной черты (корня иерархии каталогов).

- Относительное путевое имя файла

Относительное имя. Определяет местоположения файла относительно текущего каталога. Для указания относительного имени могут использоваться точки, двоеточия.

История команд

Хранится в памяти командного интерпретатора.

Перемещение по истории команд

В приглашении командного интерпретатора – стрелка вверх/стрелка вниз.

```
$ history
```

Отображение хранимой истории команд с порядковыми номерами.

```
$ history 3
```

Указание количества отображаемых команд.

```
$ !-3
```

Выполнение команды из истории: запуск третьей команды «снизу» истории.

```
$ !ls
```



Запуск команды по имени – запускается последняя команда с данным именем и с теми параметрами, с которыми она вызывалась ранее.

Поиск по истории команд

- **Ctrl+R**

Поиск по истории команд.

Постоянное хранение истории команд

- файл **.bash_history**

Хранение истории команд в домашнем каталоге пользователя. Запись производится при завершении сеанса командного интерпретатора. Чтение из файла в память – при запуске.

- **HISTSIZE**

Переменная окружения, определяет объем хранимой истории (количество команд).

- **HISTFILESIZE**

Переменная окружения, определяет объем хранимой истории (размер файла истории).

Перенаправление консольного ввода-вывода

Примеры применения

```
$ env > variables.list
```

Запись результатов выполнения в файл.

```
$ grep "bash" < /etc/passwd
```

Обработка данных из файла.

```
$ ls /etc | nl
```

Передача вывода другому процессу для обработки.



Консольный ввод-вывод



Рисунок 9: Консольный ввод-вывод процесса

Стандартные дескрипторы ввода/вывода

ID	Имя	Описание	Перенаправление
0	STDIN	Стандартный поток ввода	<
1	STDOUT	Стандартный поток вывода	>(1>) / >>
2	STDERR	Стандартный поток ошибок	2>

Пример

```
$ ls -l dir 2> err-list 1> corr-list
```

Получение STDIN из файла

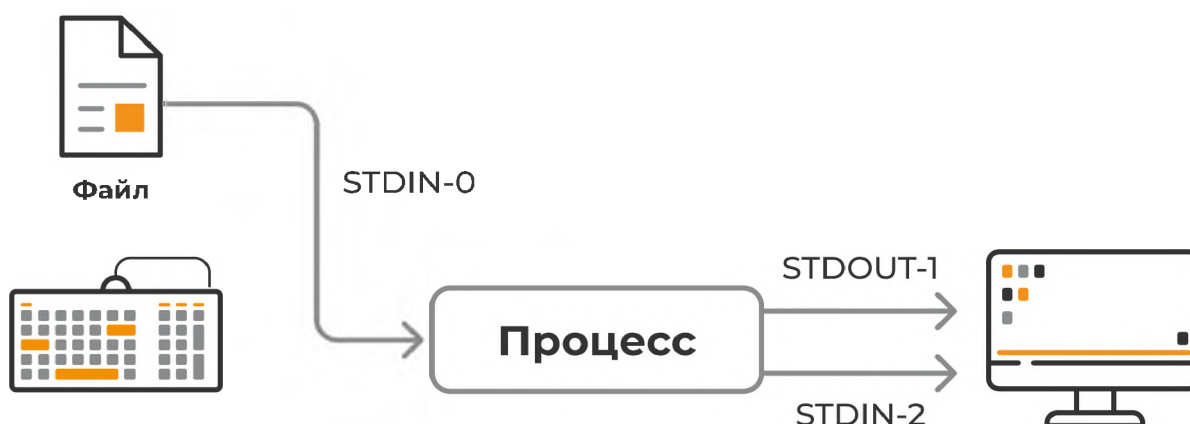


Рисунок 10: Перенаправление стандартного потока ввода процесса



Пример

```
$ sort < /etc/passwd
```

Перенаправление подстановкой содержимого файла.

```
$ grep world <(echo "hello world")
```

Перенаправление результатом вывода команды.

Перенаправление STDOUT в файл

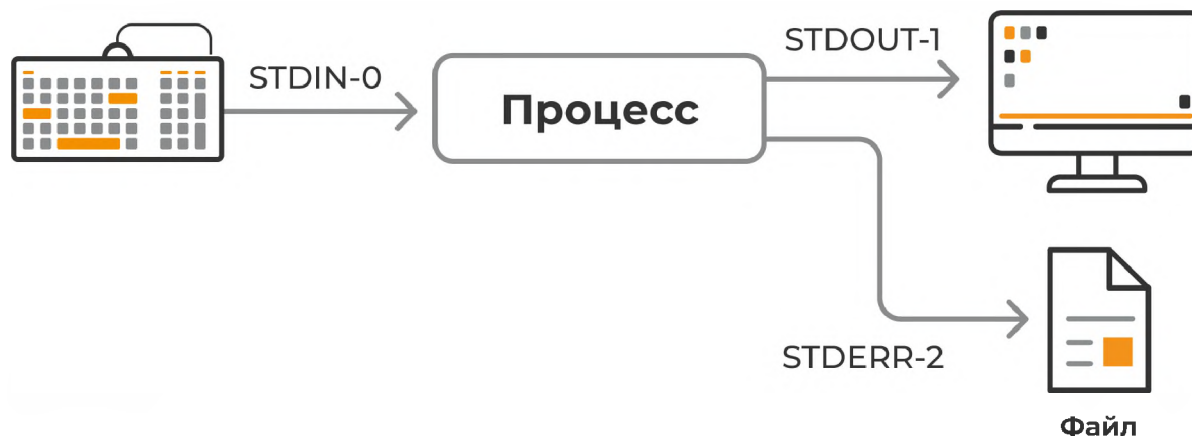


Рисунок 11: Перенаправление стандартного потока вывода в файл

Пример

```
$ echo "Line 2" >> example.txt  
$ ls -l /var/log > log.files  
$ tar -zc include > ~/include-backup.tar.gz  
$ gzip < file.txt > file.txt.gz  
$ grep -v "^#" /etc/resolv.conf > resolv2.conf
```



Перенаправление STDERR в файл

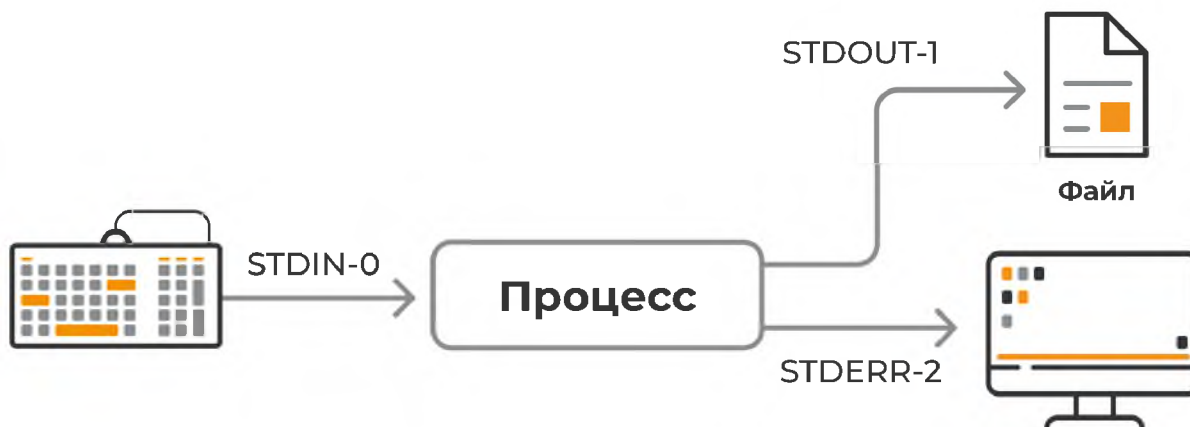


Рисунок 12: Перенаправление стандартного потока ошибок в файл

```
$ ls /fake
ls: cannot access /fake: No such file or directory
$ ls /fake > output.txt
ls: cannot access /fake: No such file or directory
$ ls /fake 2> error.txt
```

- Подавление вывода

```
$ ls /not-exist 2> /dev/null
```

Вывод в никуда – подавление потока ошибок.

Объединение потоков вывода

```
$ ls /fake /etc/ppp > example.txt 2> error.txt
```

Направляем **STDOUT** и **STDERR** в разные файлы.

```
$ ls /fake /etc/ppp &> all.txt
```

Объединяем потоки **STDOUT** и **STDERR**.

```
$ find /etc > log.txt 2>&1
```



Модуль 2

41

Основы работы в командной строке

Другой синтаксис.

Использование завершающих символов при перенаправлении

```
$ cat > file << EOF
```

```
---
```

```
---
```

```
---
```

Перенаправляется всё вплоть до символа EOF.

Каналы (конвейеры) команд (**Command Line Pipes**)

```
$ ls /etc | head  
$ ls /etc/ppp | nl  
$ cat /etc/passwd | sort  
$ grep vasya /etc/group | sort > groups.txt
```

Принцип – **STDOUT** одного процесса объединяется с **STDIN** другого процесса средствами командного интерпретатора.

```
$ ls /fake /etc/ppp 2>&1 | nl
```

Передача потока ошибок в канал – требует конструкции объединения каналов.

```
$ dd if=/dev/sda1 | pv --progress --rate | gzip -9 > sda1.gz
```

Количество объединяемых команд не ограничено.

Ветвление каналов вывода – утилита **tee**

Используется, как правило, в конвейерах выполнения команд, когда надо сохранять промежуточные результаты.

```
$ wc -l file1.txt | tee file2.txt
```



Проверит количество строк в файле file1.txt, выведет результат в терминал и сохранит его в файле file2.txt.

```
$ ls | tee test.txt | grep 'py'
```

Выведет список файлов внутри папки и с помощью первого канала запишет вывод в файл test.txt. После этого передаст вывод третьей команде – greg для идентификации файлов, содержащих в себе строку ru.

Специальные символы командного интерпретатора

Двойные кавычки (нестрогое экранирование)

```
$ mkdir Long Dir  
$ mkdir "Long Dir"
```

Показывают командному интерпретатору, что всё что заключено в кавычки должно рассматриваться как единая строка, а не как набор параметров, разделенных пробелом и спецсимволов, которые должны интерпретироваться отдельно.

Одинарные кавычки (строгое экранирование)

```
$ echo "PATH is $PATH"  
PATH is /home/egor/bin:/usr/lib/kf5/bin:/usr/local/bin:  
/usr/bin:/bin:/usr/X11R6/bin:/usr/games  
$ echo "PATH with \$ is $PATH"  
PATH with $ is /home/egor/bin:/usr/local/bin:/usr/bin:  
/bin:/usr/X11R6/bin:/usr/games  
$ echo 'PATH is $PATH'  
PATH is $PATH
```

Дополнительно к функционалу двойных кавычек блокируют подстановку переменных и подстановку выполнения команд.

Обратная косая черта

```
$ echo "PATH is $PATH"  
PATH is /home/egor/bin:/usr/lib/kf5/bin:/usr/local/bin:  
/usr/bin:/bin:/usr/X11R6/bin:/usr/games  
$ echo "PATH is \$PATH"  
PATH is $PATH
```



Позволяет экранировать одиночный символ.

Wildcard-символы командного интерпретатора

- Wildcard-символы

Шаблонные символы, раскрываемые командным интерпретатором **по содержимому каталога**.

- *

Соответствует любому количеству любых символов (0 или более) в имени файла.

- ?

Соответствует одному любому символу в имени файла

- [abc]

Подстановка символов из заданного перечня – **a,b,c**.

- [a-z]

Подстановка символов из диапазона от **a** до **z**.

- !

Отрицание в шаблонах.

Примеры

```
$ echo /etc/[gu]*
```

Все файлы из каталога **/etc**, начинающиеся с **g** или **u**.

```
$ echo /etc/*[0-9]*
```

Все файлы из каталога **/etc**, содержащие в имени цифру.

```
$ echo /etc/*[!a-t]
```

Все файлы из каталога **/etc**, заканчивающиеся не на символы от **a** до **t**.

```
$ echo /etc/[!DT]*
```



Все файлы из каталога **/etc**, не начинающиеся с **D** или **T**.

Объединение выполнения команд

Подстановка выполнения команд (command substitution)

```
$ ls -l `which date`  
$ rpm -qf $(which find)
```

Примеры использования подстановки

```
$ date  
Mon Nov 4 03:35:50 UTC 2018  
$ echo "Сегодня date"  
Сегодня date  
$ echo "Сегодня `date`"  
Сегодня Сб ноя 2 18:21:42 MSK 2019  
$ echo "Сегодня $(date)"  
Сегодня Сб ноя 2 18:21:42 MSK 2019
```

Использование подстановки вместе с экранированием.

Утилита xargs

```
$ rpm -qa | xargs rpm -V
```

Использует строки, полученные на стандартном входе как параметры следующей за ней команды.

Управляющие последовательности (символы объединения)

Позволяют выполнять несколько команд разом (последовательно). Безусловно, или в зависимости от результата выполнения предыдущей команды.

```
$ cal 1 2030; cal 2 2030; cal 3 2030
```

Символ **;** между командами – безусловное объединение.

Условные объединения

```
$ ls /etc/ppp && echo success  
ip-down.d ip-up.d
```




```
success
$ ls /etc/junk && echo success
ls: cannot access /etc/junk: No such file or directory
```

Объединение «И» – вторая команда выполняется только при успехе выполнения первой.

```
$ cd /srv/data/tmp && rm -rf *
```

Удаление всего содержимого заданного каталога.

```
$ ls /etc/ppp || echo failed
ip-down.d ip-up.d
$ ls /etc/junk || echo failed
ls: cannot access /etc/junk: No such file or directory
failed
```

Объединение «ИЛИ» – вторая команда выполняется только при неуспехе первой.

Запуск отдельного интерпретатора для выполнения группы команд

```
$ (cd /usr/include ; tar -czf $HOME/backup.tgz *)
$ (cd /srv/data/tmp && rm -rf *)
```

После выполнения группы команд восстановится начальный контекст, т.к. отдельный командный интерпретатор будет завершен.



Модуль 3 Получение документации

Справочная система man (UNIX)

Основы работы

```
$ man ls
```

Стандартная справка по внешним (с точки зрения командного интерпретатора) командам.

```
$ echo $PAGER
```

Для перелистывания страниц справки команда **man** использует **пейджер**, определяемый через значение переменной.

- **less**

Пейджер по умолчанию в ОС «Альт».

- **more**

Старая, традиционная программа-пейджер.

- клавиша **h**

Справка по работе со справкой (с пейджером *less*).

Секции man-страниц

Секция	Описание
NAME	Имя и краткое описание
SYNOPSIS	Синтаксис вызова команд
DESCRIPTION	Текстовое описание
OPTIONS	Список опций
FILES	Файлы, относящиеся к команде
AUTHOR	Автор man-страницы/программы
REPORTING BUGS	Куда отправлять ошибки
COPYRIGHT	Лицензия
SEE ALSO	Дополнительные ссылки



Описание синтаксиса вызова команд по man

- **[]**

Указывают необязательные опции/параметры команды.

- **|**

Указывает альтернативность опций – или-или, но не вместе.

- **{}**

Выбор из вариантов – один обязательно.

- Пример

```
SINOPSIS  
setfacl [-bkndRLPvh] [{-m|-x} acl_spec] [{-M|-X} acl_file] file ...
```

Пример метасинтаксической конструкции.

```
[-bkndRLPvh]
```

Опциональный набор параметров.

```
{-m|-x}
```

Один из двух вариантов на выбор.

```
{-m|-x} acl_spec
```

После одного из этих вариантов – обязательная спецификация acl.

```
[{-m|-x} acl_spec]
```

И все это, в свою очередь, опционально в командной строке.

```
file ...
```

Один файл обязательно, но может быть несколько.

Поиск по man-странице (пейджер less)

```
/string<Enter>
```



Выполнить поиск строки **string** вниз по тексту, найденные вхождения будут подсвечены.

n

Перейти к следующему вхождению строки.

/<Enter>

Повторить поиск.

?string<Enter>

Поиск вверх по тексту.

N

Перейти к следующему вхождению вверх по тексту.

?<Enter>

Перейти к предыдущему вхождению.

Разделы man-страниц

1. General(User) Commands
2. System Calls
3. Library Calls
4. Special Files
5. File Formats and Conventions
6. Games
7. Miscellaneous
8. System Administration Commands
9. Kernel Routines

LS(1)

User Commands

Указание на man-странице секции, к которой она принадлежит.



Использование man-страниц из конкретного раздела

```
$ man -f passwd
passwd (5) - файл паролей
passwd (8) - passwd wrapper
passwd (1) - обновление аутентификационных данных п...
$ whatis passwd
. . .
```

Страницы на одно и то же ключевое слово могут встречаться в нескольких секциях.

```
$ man 5 passwd
```

Указание секции.

```
$ man -a passwd
```

Последовательное отображение man-страниц во всех секциях подряд.

Поиск нужной man-страницы – man что?

Не всегда понятно «man что?». Когда знаешь ответ на этот вопрос – всё остальное – детали.

```
$ man -k copy
cp (1) - copy files and directories
cpgr (8) - copy with locking the given file to the pass..
cpio (1) - copy files to and from archives
cppw (8) - copy with locking the given file to the pass..
dd (1) - convert and copy a file
```

Поиск по именам страниц и описанию.

Справочная система проекта GNU – info

Система info

Гипертекстовая система, которую можно читать как книжку с навигацией по оглавлению и перекрестным ссылкам.

```
$ info ls
```



Модуль 3

50

Получение документации

Открытие конкретной страницы.

```
$ info
```

Открытие корня справочной системы.

Перемещение по гипертекстовой справке

Базовое перемещение по документу при помощи стрелок

- Ссылки в документе - раздел *** Menu:**

Ссылки, начинаются с символа *****; перемещение **Enter**

Клавиша	Действие
U	На начало исходной страницы
L	На последнее местоположение перед переходом
h	Описание использования
H	Справка по клавишам навигации
q	Выход

Дополнительные источники справочной информации в системе

Справка по встроенным командам

```
$ help pwd
```

Справка по встроенным командам интерпретатора.

Справка по внешним командам

```
$ ls --help  
$ ls --help | less
```

Дополнительным (основным) источником информации по параметрам и использованию команд может являться параметр – **help**.

Справка в интерактивных утилитах

```
less (h)
```



Модуль 3

51

Получение документации

```
top (h)
vim (:help)
```

Встроенная справка интерактивных инструментов.

Дополнительная справка в пакетах ПО

```
/usr/share/doc
```

Каталог с документацией из пакетов ПО.

Справочные ресурсы по ОС «Альт»

Штатная документация по дистрибутивам

```
https://docs.altlinux.org
```

Документация по техническим решениям «Базальт СПО»

- Инфраструктурные решения

```
https://docs.altlinux.org/ru-RU/index.html#domain
```

- Облачные решения

```
https://docs.altlinux.org/ru-RU/index.html#cloud
```

Wiki-страницы

```
https://www.altlinux.org/Главная\_страница
```




Модуль 4 Обработка текста

Просмотр текста

Не интерактивный просмотр текста

- **cat**

Конкатенация файлов (зачастую используется для вывода на экран).

```
$ cat [ПАРАМЕТР] [ФАЙЛ]...
```

Конкатенация – берёт все переданные параметрами файлы и выводит на терминал.

```
$ cat /etc/passwd
```

Вывод содержимого текстового файла на экран.

```
$ cat file1 file2 file3 > bigfile  
$ cat file1 file2 file3 | program
```

Удобна для перенаправления текста на другие команды для организации конвейерной обработки текста.

- См Приложение. п.2.1

- **head**

Вывод начала файла.

```
$ head /etc/passwd
```

Отображает первые несколько (head) строк текстового файла – по умолчанию 10.

```
$ head -n 3 /etc/passwd
```

Отображение указанного количества строк файла.

- См Приложение. п.2.2



- **tail**

Вывод последних строк файла.

```
$ tail /etc/passwd
```

Отображают последние несколько (tail) строк текстового файла – по умолчанию 10.

```
$ tail -f /var/log/messages
```

Интерактивный режим – отображает последние строки файла по мере их изменения – часто используется с файлами журнала.

- См Приложение. п.2.3

Построчная фильтрация текста

- **grep** – General Regular Expression Print

Поиск подстроки (регулярного выражения) в тексте.

```
$ grep bash /etc/passwd
```

Фильтрация текстовых данных (файл, стандартный вход) по шаблону. Выдает все строки, где встретился указанный шаблон.

```
$ cat /etc/passwd | grep --color -n bash
```

- См Приложение. п.2.4

Фильтрация текста по полям

- **cut**

Вырезание текста по полям.

- См Приложение. п.2.5

```
$ cat /etc/passwd | cut -d: -f1,5-7  
$ ls -l | cut -c1-11,50-
```

Пример

- **sed**



Потоковый редактор текста.

- **awk**

Табличный текстовый процессор.

Нумерации и подсчет

- Утилита **wc**

Статистика текста (Word Count) – количество строк, слов, байт.

```
$ wc /etc/passwd
28  37 1455 /etc/passwd
$ ls /etc | wc -l
142
```

Пример

- См Приложение. п.2.6

- Утилита **nl**

Нумерация строк

Интерактивное отображение текста

```
$PAGER
```

Переменная окружения, позволяющая определять используемое средство интерактивного отображения (т.е. пейджер).

- **more**

Традиционный, малофункциональный пейджер текста. Выводит текст постранично с возможностью перемещения по страницам и поиска текста.

```
$ more /etc/passwd
$ PAGER=more man useradd
```

- **less**

Современный пейджер, используемый по умолчанию в ОС «Альт» (в частности при отображении man-страниц).



```
$ less /etc/passwd
```

Интерактивный просмотр файла

Редакторы текста

Основные редакторы текста в ОС «Альт»

- **vi** – visual editor

Очень мощный редактор, но кривая обучения очень крута на старте. Для новичков интерфейс может показаться громоздким и непонятным. Установлен в ОС «Альт» и многих других ОС на основе ядра Linux «из коробки».

- **vim** – Vi IMproved

Усовершенствованная версия Vi.

- **emacs**

Еще более мощный редактор, крутая кривая обучения, избыточен для задач системного администратора. По умолчанию не устанавливается. Используется преимущественно как среда разработки или рабочая среда для работы (представляет собой комбайн с кучей расширений – встроенный почтовый клиент).

- **nano**

Золотая середина, прост в освоении, достаточен для простых правок текста, типа редактирования скриптов, конфигурационных файлов и т.п.

- **mcedit**

*Встроенный редактор консольного файл-менеджера **mc** – наиболее близок к редактированию текста а-ля из **Norton Commander/Far**.*

- **pluma**

Текстовый редактор графической среды MATE.



Использование **nano**

```
$ nano file.txt
```

Команды редактора вводятся вместе с клавишей **Ctrl**. Сводка по командам – в нижней части экрана.

Команды	Описание
^O	Сохранить, запросит подтверждение файла
^X	Выйти
^W	Поиск
^W, ^R	Поиск и замена
^Y	страница вверх
^V	страница вниз
^G	справка по командам

Редакторы **vi/vim**

```
$ vim file.txt
```

Как выжить с системе с такими редакторами. Главная особенность – два режима

- **командный** режим

Ввод команд – режим по умолчанию, клавиша **i** – переход в режим редактирования.

- режим **редактирования**

Правка текста

- подрежимы вставки и замены

Переключение между ними клавиша **Ins**, выход в командный режим – клавиша **Esc**.

- комбинации клавиш командного режима

Переход в командный режим – клавиша **ESC**, > команды начинаются с двоеточия.

- **:w**



сохранение отредактированного файла

- **:q**

ВЫХОД

- **:wq**

выход с сохранением

- **:q!**

выход без сохранения

- **:help <topic>**

в командном режиме редактора

- **vimtutor**

обучающий режим

Основы написания сценариев на языке командного интерпретатора

Общий синтаксис сценариев

```
$ cat hello.sh
#!/bin/sh
echo "Hello, world!"
```

Первая строка – специальный комментарий, начинающийся с **#!** (называется *sha-bang*), указывающий, какой интерпретатор команд необходимо вызывать для интерпретации данного сценария.

Sha-bang

```
#!/bin/sh
#!/bin/bash
#!/usr/bin/perl
#!/usr/bin/tcl
#!/bin/sed -f
#!/usr/awk -f
```



Пути должны соответствовать

Создание файла сценария

```
$ touch bf.sh  
$ nano bf.sh
```

Создаем и открываем на редактирование

```
#!/bin/sh  
tar -cJf ~/backup.tar.xz ~/work
```

Добавляем текст скрипта

Запуск сценариев

```
$ sh ./bf.sh
```

1. Передаем скрипт соответствующему интерпретатору (**bash**).

```
$ chmod +x bf.sh  
$ ./bf.sh
```

2. Делаем файл скрипта исполняемым командой **chmod** и запускаем непосредственно.

```
$ mv bf.sh bin/  
$ PATH=$HOME/bin:$PATH  
$ bf.sh
```

*При необходимости добавляем путь к скрипту в переменную окружения **PATH**.*



Модуль 5 Организация хранения данных

FHS – Filesystem Hierarchy Standard

Стандарт FHS

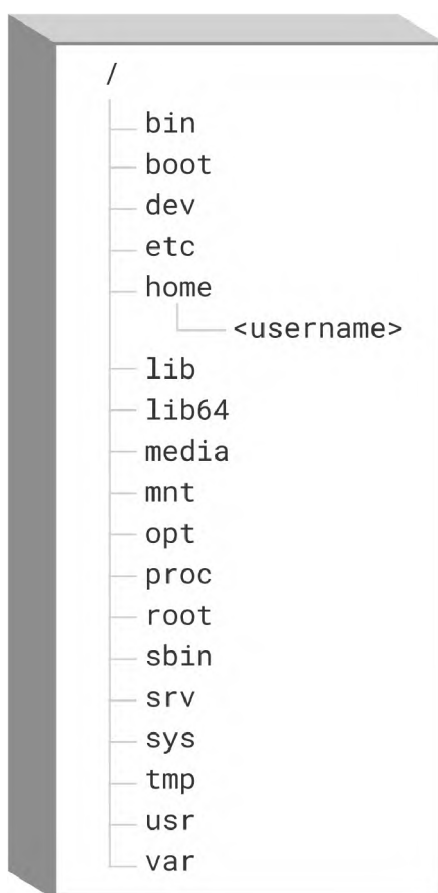


Рисунок 13: FHS

Определяет местоположение файлов исходя из их типа/назначения. Вырос из общепринятых подходов к размещению файлов в UNIX-подобных операционных системах (System V, BSD и др.). Дает ответ на вопрос, в какой части файловой системы стоит искать тот или иной файл.

- **Linux Foundation**



Модуль 5

60

Организация хранения данных

Организация, управляющая стандартом FHS.

- FHS 3.0

Актуальная версия стандарта.

Корневой каталог / и каталог /boot

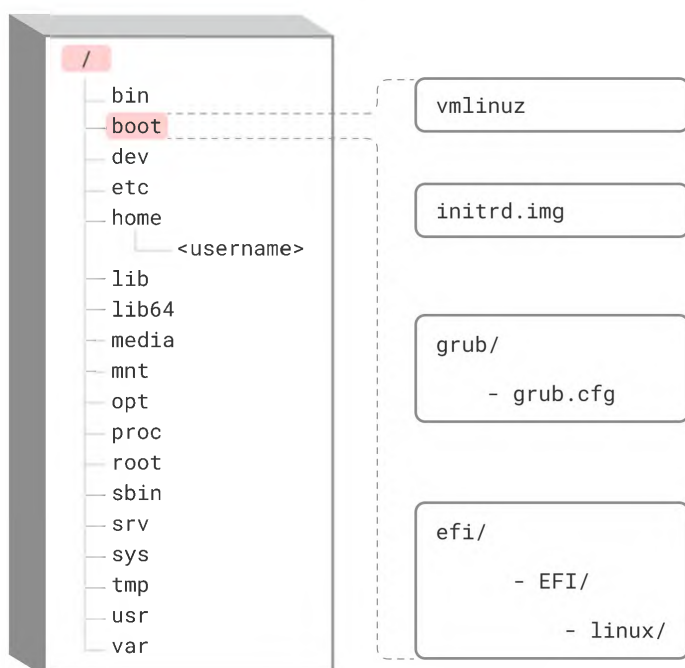


Рисунок 14: Корневой каталог / и каталог /boot

Каталоги /bin и /sbin

- **bin**

Наиболее важные пользовательские приложения.

- **sbin**

Основные **системные исполняемые файлы**, необходимые для загрузки системы, инструменты администрирования, исполняемые файлы системных сервисов.



Каталог /dev – псевдофайловая система udevfs

Размещается в памяти, заполняется при старте системы и добавлении устройств.

- **udev – Universal Device Daemon**

Управляется средствами udevd - следит за появлением устройств.

- Специальные файлы устройств

*Эти файлы являются **«точкой входа»** в драйвер устройства.*

- **символьные устройства**

Единица обмена – 1 символ, например, терминалы.

- **блочные устройства**

Единица обмена – блок данных, например, разделы диска.



Каталог /etc

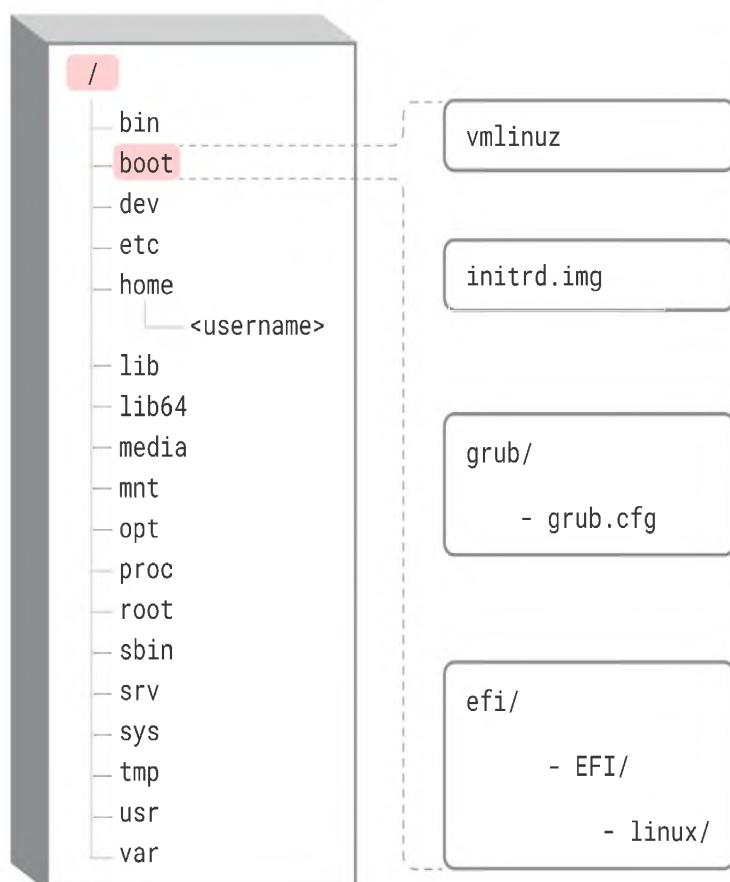


Рисунок 15: Каталог /etc

Общесистемная конфигурация

- Текстовые файлы

Двоичные файлы отсутствуют.

- Возможно редактирование

Текстовыми редакторами, но для внесения определенных изменений рекомендуется использовать специальные утилиты.

Каталоги /home и /root

- **home**



Содержит **домашние каталоги** пользователей системы. Эти каталоги создаются при добавлении пользователей. Может дополнительно структурироваться **подкаталогами по группам или доменам** пользователей. Передаются **во владение** соответствующих пользователей и доступны для записи только своим владельцам.

- **root**

Вынесенный отдельно домашний каталог суперпользователя.

Каталоги /lib, /lib32, /libx32 и /lib64

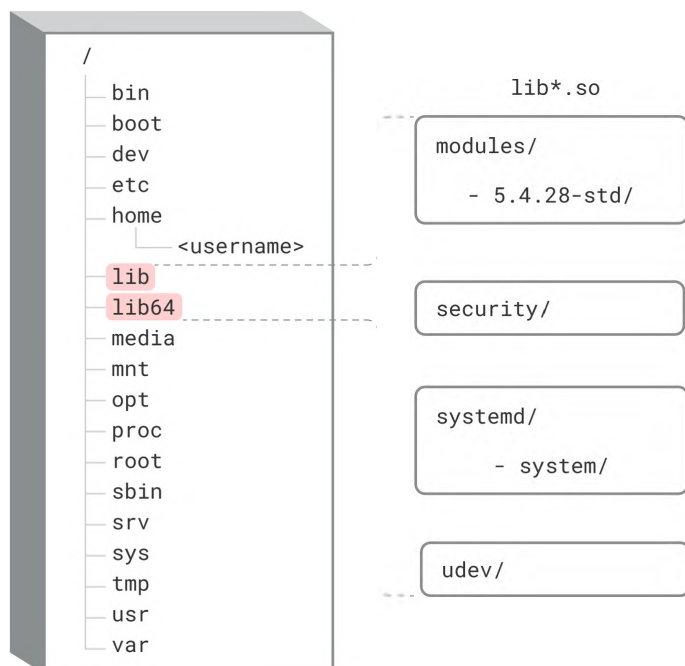


Рисунок 16: Каталоги /lib, /lib32, /libx32 и /lib64

Разделяемые библиотеки

Каталоги /media и /mnt

- **media**



Для монтирования **съёмных** носителей. В том числе для автоматического монтирования. Тенденция отказа в пользу **/run/media/[username]/...**

- **mnt**

Каталог для **временного** монтирования сетевых файловых систем (NFS, SMB) и дисковых разделов для краткосрочного использования, например, при реорганизации хранения данных.

Каталог /opt

Используется для установки сторонних программных пакетов, не поддерживающих FHS, слабо интегрированных в дистрибутив и т.п. Часто – проприетарное ПО.

```
/opt/<software_name>
```

Своя иерархия в каждом пакете. Если ПО можно развернуть по-нормальному (FHS), то лучше в **/opt не ставить**.

Каталог proc – псевдофайловая система procfs

- Псевдофайловая система

Ее данные не хранятся физически на диске

- **procfs**

Является интерфейсом к определенным структурам данных ядра ОС. Информация в виде файлов о: – процессах – нитях – используемых ресурсах – и т.п.

```
/proc/<pid>
```

Каждый процесс, работающий в системе создает в **/proc** подкаталог **/proc/[pid]**.

Каталог sys – псевдофайловая система sysfs

Информация в виде файлов об аппаратных устройствах на шинах: PCI, USB, SCSI, и т.п.



Каталог srv

Каталог для **данных**, не относящихся к конкретным пользователям. Данные, скрипты для серверов (FTP, HTTP и т.п.). **Репозитории** систем контроля версий (git и т.п.). Приложения не должны устанавливаться в /srv. Вместо **/srv** часто используются подкаталоги **/var – /var/www/html** и т.п.

Каталог tmp

Хранилище **временных** файлов. Все содержимое можно безболезненно **удалять** (лучше удалять только старые файлы).

- Псевдофайловая система в ОЗУ (RAM-диск)

В ОС «Альт Рабочая станция»

Каталог usr

Каталог для файлов, не требующихся при загрузке системы: прикладное ПО, дополнительные библиотеки и т.п. Отдельная иерархия, очень похожая на корневую.

Каталог	Описание
/usr/bin	Дополнительное ПО
/usr/etc	Дополнительные настройки
/usr/games	Данные игр
/usr/include	Заголовочные файлы приложений
/usr/lib	Файлы библиотек
/usr/lib64	Файлы библиотек 64-bit
/usr/local	Третья иерархия для локального ПО
/usr/sbin	Дополнительное системное ПО
/usr/share	Файлы данных только для чтения
/usr/src	Исходники и заголовочные файлы ядра
/usr/tmp	Второй временный каталог

Каталог var

Содержит изменяемые при работе системы файлы

/var/log



Модуль 5

66

Организация хранения данных

Файлы журнала

`/var/spool`

Спулинг различных служб

`/var/cache`

Кэш различных служб

`/var/mail`

Каталоги с почтой

Каталог run

Псевдофайловая система в ОЗУ. Для рабочих временных файлов (механизмы взаимодействия процессов между собой и др.).

`/run/media/[username]/`

Автоматическое монтирование съемных носителей.

Работа с дисковыми разделами

Разделы дисков

- Разделы – **partitions**

(они же логические диски в Windows) позволяют эффективно организовать дисковое пространство

- Файлы блочных устройств

обеспечивают низкоуровневый/системный интерфейс взаимодействия с накопителями

- Именованние разделов

Цифры по порядку после имени диска

```
$ ls -l /dev/sd*  
brw-rw---- 1 root disk 8,  0 Jun 24 06:13 /dev/sda
```



```
brw-rw---- 1 root disk 8,  1 Jun 24 06:13 /dev/sda1
brw-rw---- 1 root disk 8,  2 Jun 24 06:13 /dev/sda2
brw-rw---- 1 root disk 8, 16 Jun 24 06:13 /dev/sdb
brw-rw---- 1 root disk 8, 32 Jun 24 06:13 /dev/sdc
```

Таблицы разделов

- **MBR** – Master Boot Record

компьютеры с BIOS – унаследован еще с времен DOS – максимум 4 первичных раздела, один можно сделать расширенным и создать до 4 разделов внутри него (итого 7).

- **GPT** – GUID Partitioning Tabel

компьютеры с EFI, максимум 2 Tb на раздел, 128 разделов.

Таблица разделов Master Boot Record – MBR

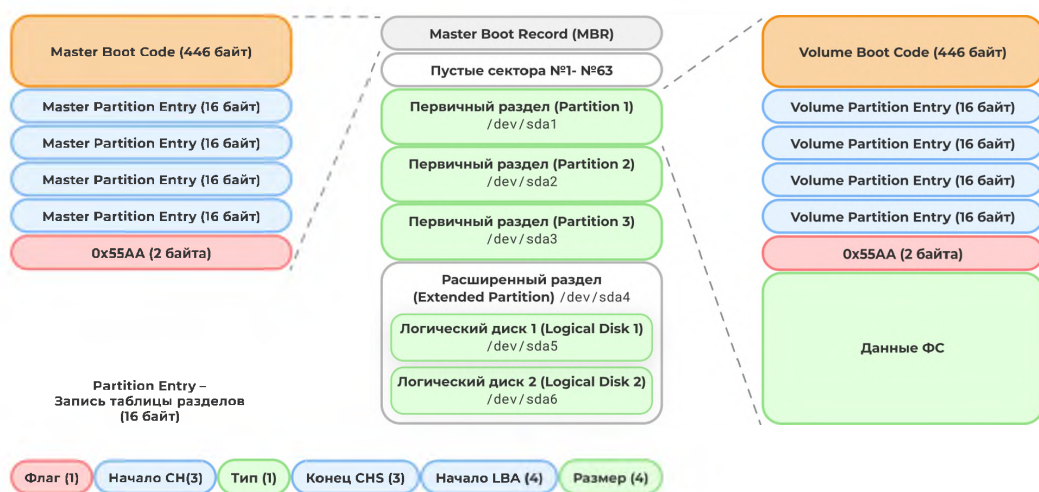


Рисунок 17: Структура MBR

- Максимальная совместимость

Используется на персональных компьютерах со времен DOS

- Встроенные ограничения

До 7 разделов, раздел до 2.1 Tb, при повреждении первых секторов, диск перестает читаться.



Таблица разделов GUID Partition Table – GPT

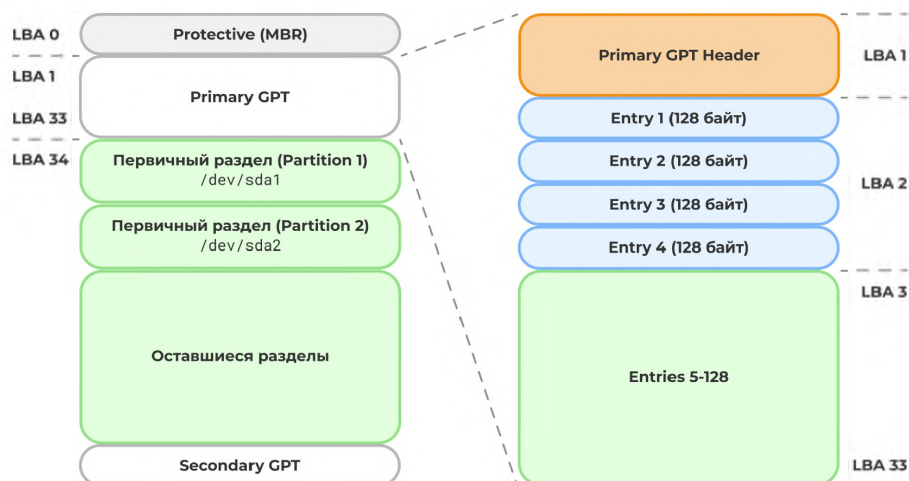


Рисунок 18: Структура GPT

- Разделы до 9,4 ЗБ

Используется блочная адресация **LBA**, вместо CHS в MBR, что позволяет создавать разделы до 9,4 ЗБ.

- Код загрузчика – в отдельном разделе

Является частью стандарта **EFI – Extensible Firmware Interface** (замена устаревшего BIOS) В отличие от MBR Не содержит **кода загрузчика**, рассчитывает, что этим будет заниматься EFI – там предусматривается на это специальный раздел.

- Дублирование таблицы разделов

Таблица разбиения **дублирована** – в начале диска и в конце.

- 128 разделов

Количество разделов на диске.

Просмотр дисковых разделов

```
$ lsblk /dev/sda
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda          8:0    0 931,5G  0 disk
├─sda1       8:1    0   260M  0 part /boot/efi
└─sda2       8:2    0    16M  0 part
```



└sda3	8:3	0	202,1G	0	part	/srv
└sda4	8:4	0	77,5G	0	part	/

- **lsblk**

Отображение блочных устройств – всех, или конкретного диска, древовидная форма вывода информации об имеющихся блочных устройствах.

- **blkid**

Показывает атрибуты блочного устройства.

Утилиты для работы с дисковыми разделами

- **fdisk**

Стандартная утилита в виде системы меню.

- **cfdisk**

Меню в текстовом интерфейсе.

- **parted**

Альтернативная утилита проекта GNU – интерактивно, – параметрами командной строки. Позволяет менять размеры разделов.

- **gparted**

GUI-версия parted, часто встраивается в инсталлятор дистрибутива Linux.



Модуль 5

70

Организация хранения данных

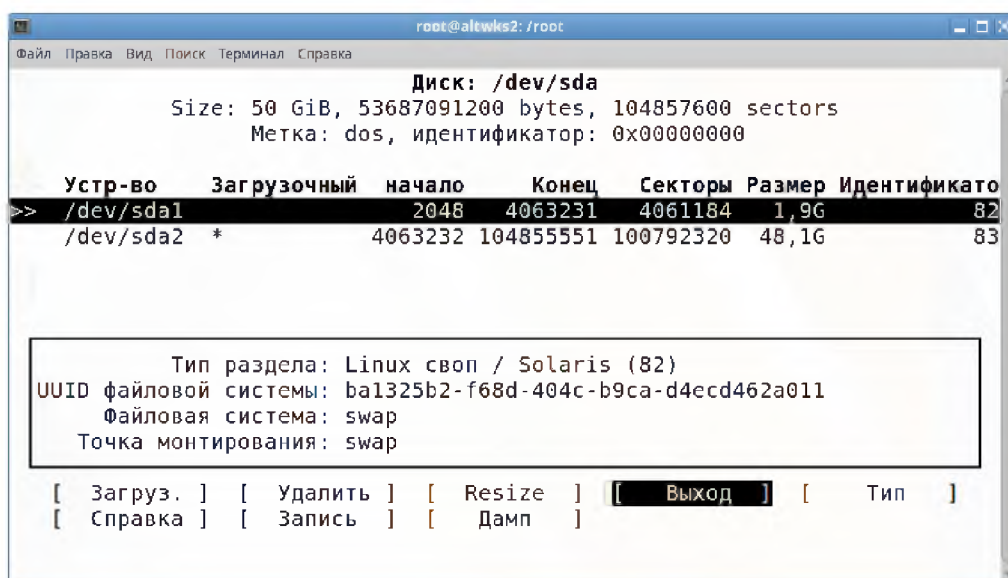


Рисунок 19: Разбиение диска – cfdisk

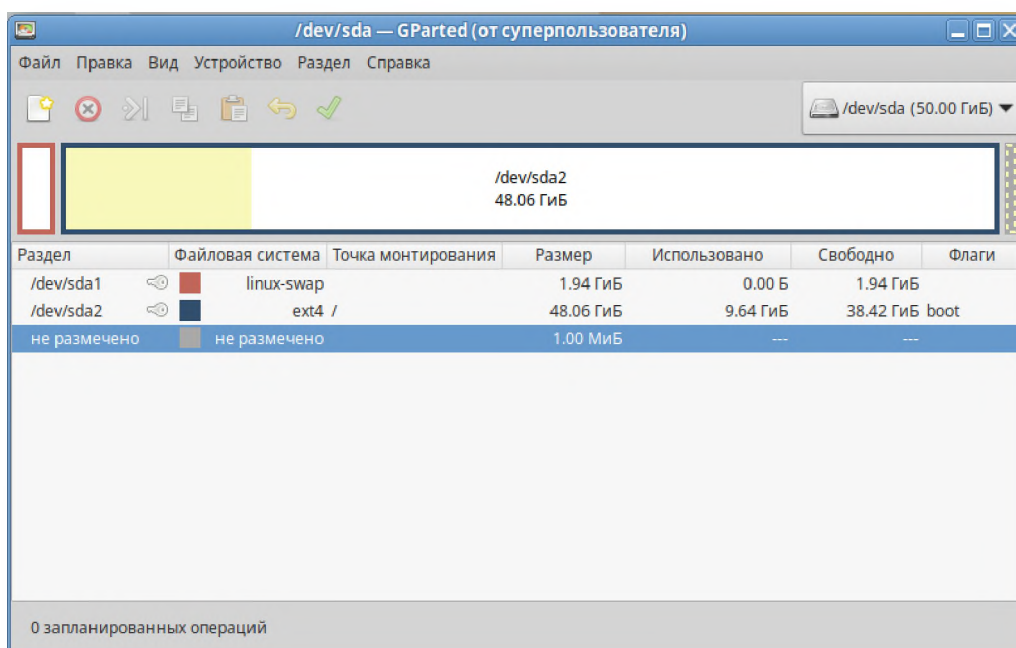


Рисунок 20: Разбиение диска – gparted

Использование fdisk

```
$ fdisk /dev/sda
```

Редактирование таблицы разбиения диска.



- Команды меню:

Команда	Действие
m	Отобразить справку
p	Отобразить таблицу разделов
n	Создать новый раздел
d	Удалить раздел
t	Изменить тип раздела
w	Записать изменения и выйти
q	Выход без изменений

Использование **parted**

```
$ parted [options] device [command]
```

Редактирование таблицы разбиения диска.

- Опции:
 - **-h** – справка
 - **-s** – скриптовый режим, без запросов к пользователю
 - **-i** – интерактивный режим (по умолчанию)

В интерактивном режиме команды вводятся интерактивно, в скриптовом - в параметрах

Команда	Действие
print	Отображение разделов
select	Выбор другого устройства
mktable	Создать таблицу разделов
mkpart	Создать раздел
name	Задать метку
set	Установить флаг
resizepart	Изменить размеры раздела
rm	Удалить раздел
rescue	Попытаться восстановить раздел
quit	Выход

```
$ parted /dev/sdb
(parted) mktable gpt
(parted) mkpart primary ext3 0 2000M
(parted) name 1 vardata
(parted) set 1 boot on
(parted) mkpart primary xfs 2000M -0M
(parted) name 2 homedata
(parted) print
(parted) quit
```



Файловые системы в ОС «Альт»

Многообразие поддерживаемых файловых систем

- https://www.altlinux.org/РазбиениеДиска#Файловые_системы

```
$ cat /proc/filesystems
```

- Дисковые ФС:
 - магнитные/твердотельные диски:
 - «родные»: **ext2, ext3, ext4**
 - «заимствованные»: **XFS(SGI), JFS(IBM)**
 - «дополнительные»: **ReiserFS, Reiser4, btrfs, zfs**
 - «для совместимости»: **FAT12, FAT16, FAT32, VFAT, NTFS**
 - оптические диски: **ISO9660, udf**
- Сетевые ФС: **nfs, smb/cifs, coda, afs**
- Псевдофайловые:
 - интерфейс с ядром: **devfs, procfs, sysfs**
 - в памяти(RAM-диск): **tmpfs**
- Внеядерные (через модуль ядра fuse): **fuse.archivemount, fuse.sshfs, fuse.encfs, fuse.fuseiso**

Создание ФС (**mkfs**)

```
# mkfs [-t fstype] [options] [device-file]
# mkfs.fstype [options] [device-file]
```

Создание файловой системы производится командой mkfs.

- **[device-file]**

файл устройства раздела диска, например, /dev/sda1.

```
# mkfs -t ext4 /dev/sda10
# mkfs.ext4 /dev/sda10
```

- Опции

Индивидуальны для каждой ФС

- **-b <size>**

Размер блока



Монтирование разделов

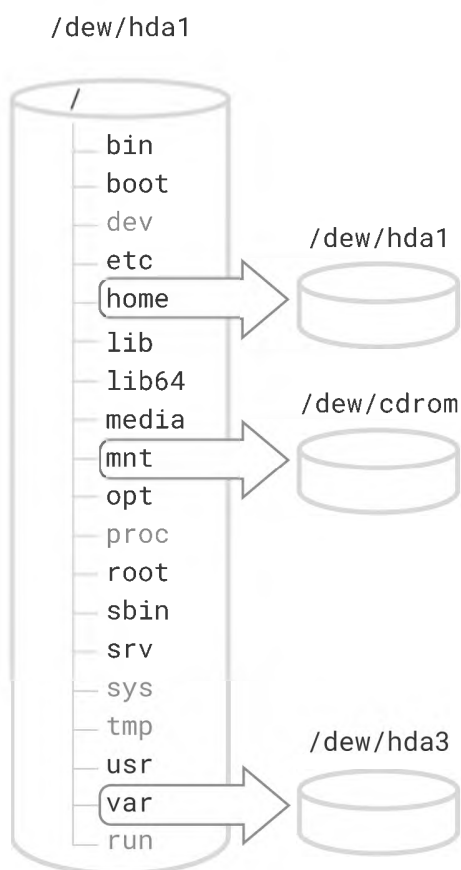


Рисунок 21: Монтирование и иерархия каталогов

- Монтирование

Подключение файловой системы в иерархию каталогов.

- Точка монтирования

Каталог подключения.

```
# mount [-t type ] [-o options] <source> <directory>
```

Монтирование: Производится в определенный каталог, за которым данная ФС будет доступна в общем дереве VFS.



```
# mount -t ext4 /dev/sdb4 /mnt
```

- /dev/sdb4

Файл устройства раздела диска.

- /mnt

Каталог монтирования, все что там было будет недоступно до отмонтирования.

- ext4

Тип файловой системы.

```
$ man mount
```

Справка

```
# umount [device-file | mount-point]
```

Отмонтирование

```
# umount /mnt  
# umount /dev/sda3
```

Утилита **df**

```
$ df [OPTIONS]
```

Отображает использование файловой системы (по умолчанию в К-байтах).

```
$ df -hT | grep ext  
/dev/sda6 ext4 692G 158G 500G 24% /
```

Пример

- См. Приложение, п.3.1

Автомонтирование – **/etc/fstab**

- Файл **/etc/fstab**



Модуль 5

75

Организация хранения данных

Содержит информацию о файловых системах, их стандартные точки монтирования и опции монтирования.

1. Ресурс монтирования

(имя файла, метка или UUID)

2. Точка монтирования

Куда

3. Тип

файловой системы

4. Список опций

через запятую

5. dump

Делать ли резервную копию утилитой dump/restore

6. pass

Порядковый номер проверки для fsck, 0 – не проверять, 1 – для корневой ФС, 2 – для остальных

```
/dev/sda6 / ext4 defaults 1 1
```

Пример записи

```
UUID=a3802... / ext4 relatime 1 1
UUID=df3a3... /srv ext4 relatime 1 2
UUID=F025-... /boot/efi vfat iocharset=utf8 1 2
tmpfs /tmp tmpfs nosuid 0 0
/swapfile none swap defaults 0 0
```

Пример файла

Дополнительные инструменты

```
e2fsck, fsck.*
```

Проверка ФС



Модуль 5

76

Организация хранения данных

```
tune2fs, tuneefs.*
```

Настройка параметров ФС

```
resize2fs
```

Изменение размера

```
truncate
```

Изменение размера файлов

```
$ truncate -s 1G trunc-image.img
```

Создание файла размером 1Г

```
dd
```

Поблочное копирование



Модуль 6 Управляющие конструкции командного интерпретатора

Переменные в скриптах на языке командного интерпретатора

Рабочий пример сценария

```
#!/bin/sh
tar -cJf ~/backup.tar.xz ~/work
```

Работа с переменными в скрипте

```
VAR=va l
```

БЕЗ ПРОБЕЛОВ В ПРИСВАИВАНИИ!

```
$VAR
```

Получение значения переменной

- **read**

Чтение значения переменной от пользователя

Пример – переменные в скрипте

```
$ cat bf.sh
#!/bin/sh
BACKUP_DIR=~/.work
BACKUP_FILE_TMPL=backup
tar -cJf $HOME/$BACKUP_FILE_TMPL.tar.xz $BACKUP_DIR
```

Описываются и используются ровно точно также, как локальные переменные/переменные окружения.

Позиционные параметры

Приём позиционных параметров

Аргументы, передаваемые скрипту из командной строки



Модуль 6

78

Управляющие конструкции командного интерпретатора

- **\$0**

Название файла сценария

- **\$1**

Первый аргумент

- **\$2, \$3**

Второй, третий и так далее...

- **\${10}, \${11}, \${12}**

Аргументы, следующие за \$9, должны заключаться в фигурные скобки, например:

- **\$***

Все позиционные параметры (в строку)

- **\$@**

Все позиционные параметры (в столбик)

- **\$#**

Количество позиционных параметров

Пример – приём позиционных параметров

```
$ cat bf.sh
#!/bin/sh
B_FILE_TMPL=backup
tar -cJf $HOME/$B_FILE_TMPL-$1.tar.xz $1
```

Обработка позиционных параметров

```
$ ./bf.sh work
```

Запуск

Обработка позиционных параметров

Пример – обработка позиционных параметров

```
$ cat bf.sh
```



```
#!/bin/sh
TAR_CMD='tar -cJf'
B_FILE_TMPL=backup
F_NAME=$B_FILE_TMPL-`echo $1 | tr '/' '-'.tar.xz
$TAR_CMD $HOME/$F_NAME $1
```

Задать команду архивирования как переменную – удобнее менять. Избавиться от символов / в имени файла архива – чтобы архивировать не только то, что в текущем каталоге.

Управляющие конструкции условного выполнения

Конструкция **if-then-else**

- Конструкция условного выполнения

```
if условие; then команды; fi
```

В одну строчку

```
if условие; then
    команды
else
    команды
fi
```

В несколько (обычно в скриптах)

- условие

Задается в виде команды, результат выполнения которой проверяется.

```
$ if touch /tmp/file1; then echo success; fi
```

Команда **test** – описание условий

```
$ test "str1" = "str2"
$ test "str1" != "str2"
```

*Сравнение строк. Возвращает **true**, если условие верно и **false**, если нет.*



Пример конструкции условного выполнения с командой **test**

```
if test "$1" != "" ; then
    команды
else
    echo "Нет параметра!"
fi
```

Проверка файлов

Синтаксис	Описание
test -f file	Верно, если файл существует
test ! -f file	Верно, если файла нет
test -d dir	Верно, если каталог существует
test -x file	Верно, если файл исполнимый

Команда test – описание условий проверки для файлов

Сравнение чисел

Синтаксис	Описание
test \$A -eq \$B	Равенство
test \$A -ge \$B	Больше или равно
test \$A -le \$B	Меньше или равно
test \$A -gt \$B	Больше
test \$A -lt \$B	Меньше
test \$A -ne \$B	Не равно

Сокращенный синтаксис **test**

```
[[  ]]
```

*Типичный вариант – использование с **if** квадратных скобок это то же самое, что команда **test**.*

```
if [[ условие ]]; then
    команды
else
    команды
fi
```

Пример – добавляем конструкцию условного выполнения

```
$ cat bf.sh
#!/bin/sh
TAR_CMD='tar -cJf '
B_FILE_TMPL=backup
```



```
F_NAME=$B_FILE_TMPL-`echo $1 | tr '/' '-'.tar.xz
if [[ "$1" != "" ]]; then
    $TAR_CMD $HOME/$F_NAME $1
else
    echo "Usage: " $0 " dir"
fi
```

Условное выполнение по переданным параметрам.

Оператор **case**

```
case переменная in
    значение 1.1|значение 1.2)
        команды 1;;
    значение 2)
        команды 2;;
    значение 3)
        команды 3;;
esac
шаблон *)
```

Конструкции циклов

Синтаксис цикла **for**

Перебор последовательности значений из списка

```
for var in list; do команды; done
for var in list; do
    команды
done
for var in list
do
    команды
done
```

Используем, чтобы обрабатывать несколько каталогов за раз.

Пример – цикл по позиционным параметрам

```
$ cat bf.sh
#!/bin/sh
TAR_CMD='tar -cJf'
B_FILE_TMPL=backup
if [[ "$1" != "" ]]; then
    for bdir in $1 $2 $3 $4 $5 $6 $7 $8 $9; do
        F_NAME=$B_FILE_TMPL-`echo $bdir | tr '/' '-'.tar.xz
```



```
    $STAR_CMD $HOME/$F_NAME $bdir
done
else
    echo "Usage: " $0 " dir"
fi
```

Цикл по позиционным параметрам

- Далее, подстановка даты

```
$ cat bf.sh
#!/bin/sh
TAR_CMD='tar -cJf'
B_FILE_TMPL=backup-$(date +%Y-%M-%d)
if [[ "$1" != "" ]]; then
    for bdir in $1 $2 $3 $4 $5 $6 $7 $8 $9; do
        F_NAME=$B_FILE_TMPL-`echo $bdir | tr '/' '-'`.tar.xz
        $TAR_CMD $HOME/$F_NAME $bdir
    done
else
    echo "Usage: " $0 " dir"
fi
```

Добавляем дату

Цикл **for** со счетчиком

```
for ((i=1; i<=N ; i++))
do
    команда
done
```

Цикл **while**

```
counter=0
while [[ $counter -le N ]]
do
    echo $counter
    (( counter++ ))
done
```



Модуль 7 Файлы и их атрибуты

Типы файлов

Системные типы файлов

```
$ ls -l
-rw-r--r-- 1 root root 15322 Dec 10 21:33 alt.log
drwxr-xr-x 1 root root 4096 Jul 19 06:52 apt
```

*Расшифровка длинного вывода **ls***

Символ	Описание
-	Обычный файл
d	Каталог
l	Символическая ссылка
b	Блочное устройство (/dev)
c	Символьное устройство (/dev)
p	Именованный канал
s	Сокет

*По выводу **ls***

Утилита **file**

Отображает тип содержимого указанного файла.

```
$ file astrafonts.tex
astrafonts.tex: UTF-8 Unicode text
$ file images/fhs-bin.png
images/fhs-bin.png: PNG image data, 211 x 381,
8-bit/color RGBA, non-interlaced
```

В ОС Linux расширение файла не несет НИКАКОЙ смысловой нагрузки.

Утилиты для работы с файлами

Копирование файлов

```
$ cp [OPTIONS] [src] [dst]
```




Примеры

```
$ cp /etc/hosts ~  
$ cp /etc/hosts ~/hosts.copy
```

Копирование и копирование со сменой имени.

```
$ cp -r /var/data /var/backup
```

Рекурсивное копирование.

```
$ cp -rn /etc/skel/. * ~
```

Использование символов подстановки при копировании.

- См. Приложение, п.4.2.

Перемещение и переименование файлов

```
$ mv [OPTIONS] [src] [dst]
```

Перемещение, а также переименование.

```
$ mv ./hosts Videos  
$ mv -v text.txt myfile.txt
```

- См. Приложение, п.4.3.

Создание файлов

```
$ touch file1  
$ ls -l file1  
-rw-r--r-- 1 user user 0 окт 29 19:42 file1
```

*Команда обновляет у файла **дату создания и модификации** на текущую. Если файл не существует, то **создает** его.*

- См. Приложение, п.4.1.

```
$ cat > file2  
Ctrl+D
```

Возможно начальное заполнение файла.



Модуль 7

85

Файлы и их атрибуты

Удаление файлов

```
rm [OPTIONS] [FILE]
```

Удаляет файлы/каталоги.

```
$ rm -rf /  
$ rm -i *.txt
```

- См. Приложение, п.4.4.

Работа с каталогами

```
$ mkdir workdata  
$ ls  
workdata
```

Создание каталога.

```
$ rmdir workdata
```

Удаление пустого каталога.

```
$ rm -r workdata
```

Удаление каталога с содержимым.

Утилита **du**

```
du [ПАРАМЕТР]... [ФАЙЛ]...
```

Отображает занятое содержимым каталогом место.

- См. Приложение, п.4.7.

Индексные дескрипторы

Хранение файлов на ФС

- Запись в каталоге

*Запись в каталоге – содержит имя файла и номер индексного дескриптора (ссылку на дескриптор). Фактически запись о файле в каталоге – это ссылка на **inode**.*



- Индексный дескриптор (inode)

Метаданные файла – содержат всю описательную информацию о файле и ссылки на блоки с данными.

- Блоки с данными файла

Непосредственно полезные данные.

```
$ ls -il /bin/ls
2753245 -rwxr-xr-x 1 root root 137888 янв 14 15:17 /bin/ls
```

- Ссылки на **inode**

*Может быть несколько. Первая ссылка создается при создании файла (вместе с самим inode). Дополнительные можно создавать командой **ln**.*

- Жёсткие ссылки

Записи в каталогах с различными путевыми именами, ссылающимися на один и тот же inode.



Содержимое inode

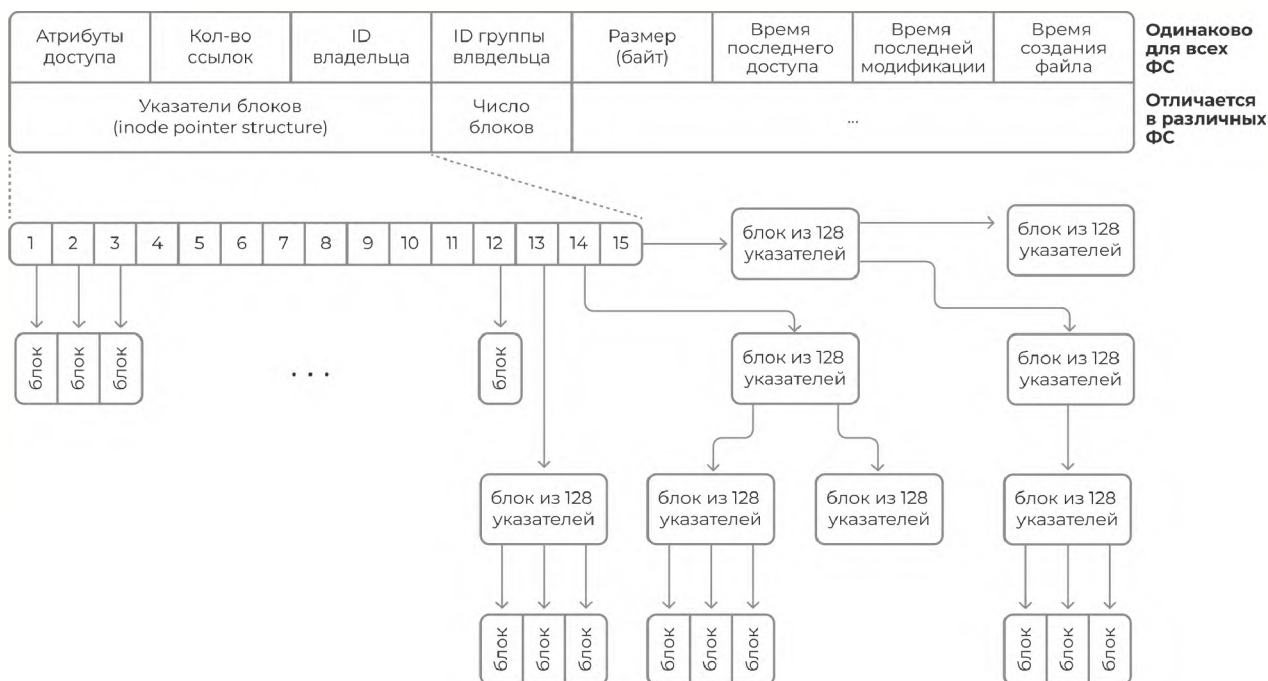


Рисунок 22: Структура inode

- **inode** (index node)

Индексный дескриптор. Содержит метаинформацию файла. Хранятся в специальной области файловой системы, часто ограниченной по размеру.

- Удаление файла

Это удаление его метаданных (**inode**).

Просмотр индексного дескриптора

- Команда **stat**

```
$ stat /etc/passwd
  файл: /etc/passwd
  Размер: 2737  Блоков: 8  Блок В/В: 4096  обычный файл
Устройство: fd00h/64768d  Inode: 1575711  Ссылки: 1
Доступ: (0644/-rw-r--r--)  Uid: (0/root)  Gid: (0/root)
Доступ: 2019-06-30 22:48:12.626850670 +0300
Модифицирован: 2019-06-28 20:59:17.394701734 +0300
Изменён: 2019-06-28 20:59:17.494701738 +0300
```



Каталоги



Рисунок 23: Структура каталога

Каталоги – это файлы, содержащие внутри **таблицу записей** вида:
– имя файла – индексный дескриптор

```
$ ls -li /home
итого 4
1703938 drwxr-xr-x 21 stud stud 4096 июл  1 17:27 stud
```

В выводе команды **ls -l** обозначаются символом **d**.

Жёсткие ссылки

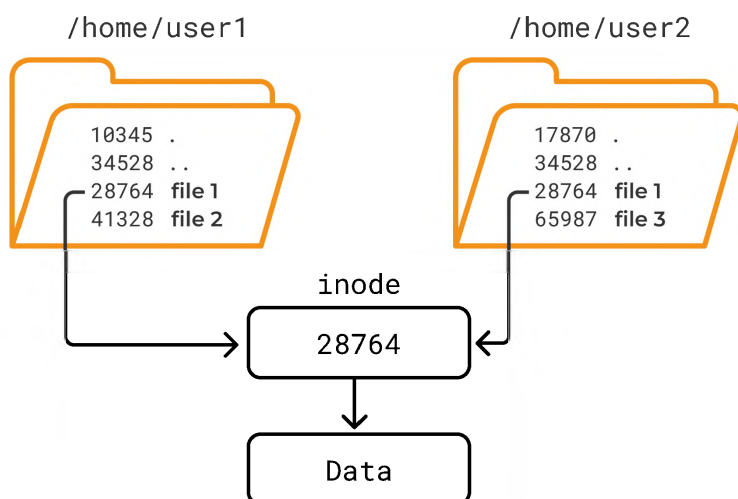


Рисунок 24: Жёсткие ссылки

```
$ ln first second
$ ln /home/user1/file1 /home/user2/file1
```



Записи в двух каталогах указывают на один и тот же *inode*.

- **Равноправие** ссылок

Ссылки **равноправны**. Изменения одного файла меняют и другой, так как это **один и тот же файл**.

- **Удаление** ссылок

Удаление первой ссылки уменьшает счётчик ссылок в метаданных, удаление второй – удаляет **метаданные (inode)**.

- **Перемещение** жёсткой ссылки

На *inode* будет ссылка из другого каталога.

- **Копирование** жёсткой ссылки

Создается новый файл (новые метаданные и данные). В старом ссылок столько сколько было, в новом – 1.

- Ограничения

Только в рамках **одной ФС**

Мягкие ссылки

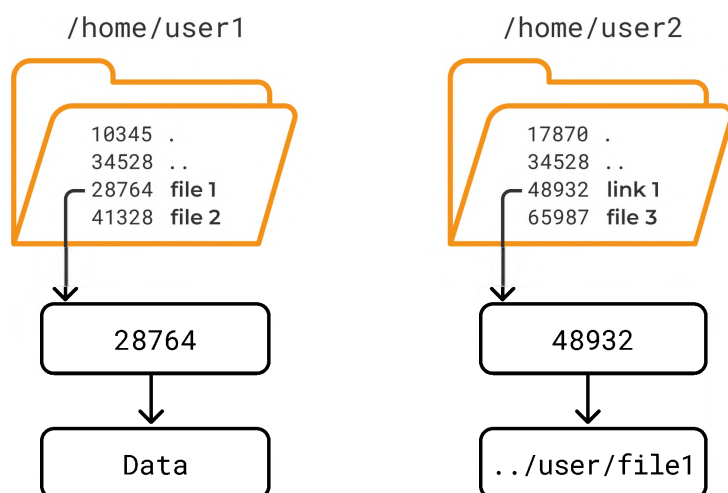


Рисунок 25: Мягкие ссылки

```
$ ln -s ../user1/file1 link1
```



«Мягкая» ссылка – файл, содержащий в своих данных путь (абсолютный или относительный) к имени другого файла. Работает в рамках всей иерархии файлов. Может содержать как относительный, так и абсолютный путь.

- Копирование и перемещение ссылки

Мягкую ссылку можно перемещать и копировать, она продолжает указывать на файл (если абсолютная ссылка).

- Перемещение целевого файла

При перемещении файла, ссылка перестает указывать на него.

- Ограничения

Не ограничен одной файловой системой. Работает в рамках всей иерархии файлов.

Сравнение мягких и жёстких ссылок

- По области действия

Жёсткие ссылки действуют только в рамках одной ФС. Мягкие ссылки могут ссылаться на другие ФС.

- По действию удаления файла

Жёсткие ссылки все равноправны – нет слабого места – файл (метаданные) существуют, пока на них есть хоть одна ссылка. Мягкие ссылки имеют слабое место – при удалении файла, на который ссылается ссылка, ссылка теряет смысл.



Модуль 8 Учётные записи пользователей и групп

Пользовательские учётные записи

Назначение учётных записей пользователей

- Учётная запись

Ключевой элемент системы разграничения доступа ОС. Пользователь, выполнив аутентификацию, представляется в системе в виде учётной записи. Идентификатор учётной записи пользователя, а также список групп, в которые пользователь входит становятся атрибутами запускаемых пользователем процессов, тем самым представляя пользователя в системе и определяя для процессов их разрешения доступа.

- Доступ к ресурсам

ОС предоставляет или отказывает процессу в **доступе к ресурсам** исходя из пользовательских идентификаторов процесса.

- Домашний каталог

УЗ связаны с **домашним каталогом** пользователя и **«профилем»** – группой индивидуальных настоечных файлов, обеспечивая индивидуальное приватное пространство для работы пользователя.

Именование пользователей

- первый символ

Подчеркивание или латинская строчная буква a-z

- дальнейшие символы

Цифры, латинские буквы, подчеркивание или дефис

- последний символ

Не должен быть дефисом

- длина имени

Поддерживаются идентификаторы до 32 символов



Иерархия УЗ пользователей

- **Суперпользователь (UID=0)**

Учётная запись с нулевым идентификатором, обладает абсолютными полномочиями в системе.

- **Служебные** учётные записи ($0 < \text{UID} < \text{UID_MIN}$)

Учётные записи, предназначенные для работы служб и прочих системных процессов, не предполагают возможности интерактивного входа в систему.

- **Операторские** учётные записи ($\text{UID_MIN} \leq \text{UID} < \text{UID_MAX}$)

Учётные записи обычных пользователей.

БД учетных записей – **/etc/passwd**

Каждая строка – запись о пользователе, поля записи разделяются двоеточием

```
login : password : UID : GID : GECOS : home : shell
```

Поля /etc/passwd

```
beav:x:1000:1000:Theodore Cleaver:/home/beav:/bin/bash
warden:x:1001:1001:Ward Cleaver:/home/warden:/bin/bash
dobie:x:1002:1002:Dobie Gillis:/home/dobie:/bin/bash
```

Пример записей /etc/passwd

- **login**

Символический идентификатор пользователя

- **password**

Поле пароля – не используется

- **UID**

Идентификатор пользовательской УЗ

- **GID**

Идентификатор пользовательской приватной группы



- **GECOS**

Описание/комментарий

- **home**

Домашний каталог пользователя

- **shell**

Командный интерпретатор по умолчанию

Инструменты для работы с пользовательскими УЗ

- **useradd** – см. Приложение п.10.1

Создаёт новую учётную запись.

- **newusers**

Создание УЗ пользователей в пакетном режиме.

- **usermod** – см. Приложение п.10.2

Изменяет данные учётной записи.

- **userdel**

Удаляет существующую учётную запись.

Примеры создания пользовательских УЗ

```
$ useradd dexter
```

Создание пользователей с настройками по умолчанию – домашний каталог, членство в группах, командный интерпретатор.

```
$ useradd -s /bin/csh -m -k /etc/skel -c "Bullwinkle J Moose" -G wheel bmoose
```

Явно указанные параметры при создании пользователя.

```
/etc/login.defs  
/etc/default/useradd
```

Параметры по умолчанию.



Модуль 8

94

Учётные записи пользователей и групп

Создание пользователей средствами ЦУС

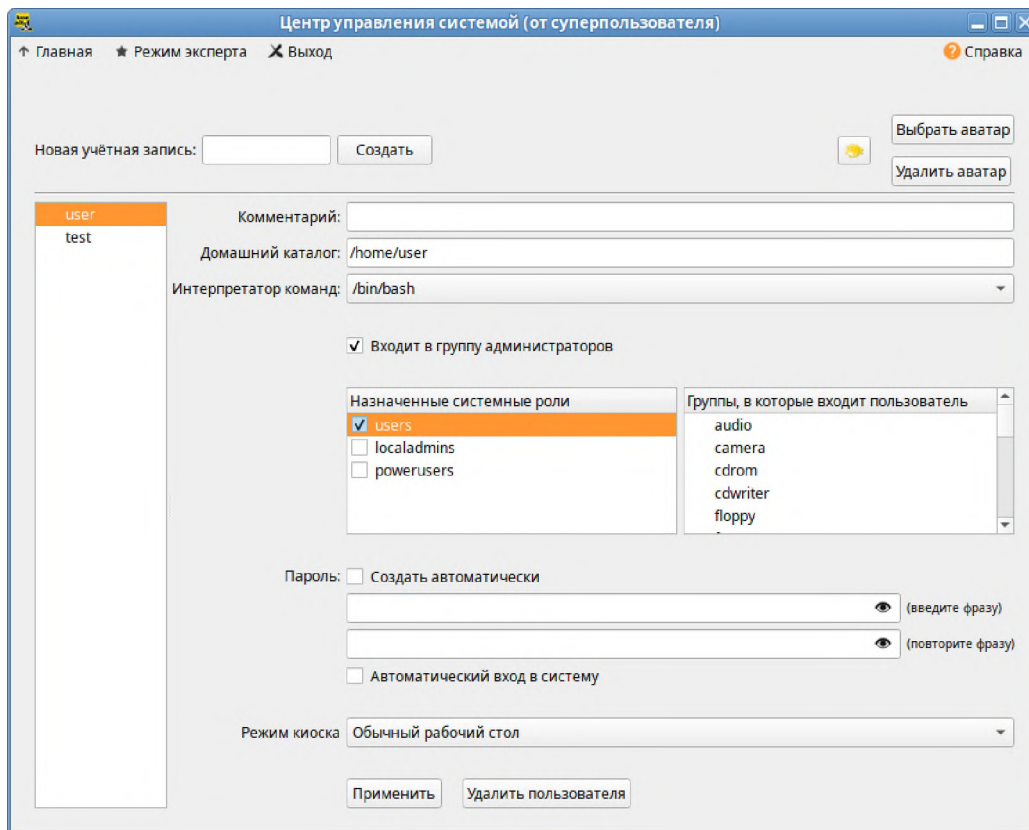


Рисунок 26: Модуль «Локальные учетные записи»

База паролей пользователей

Формат записи о пароле пользователя

```
login:hash:LAST_DAY:MIN_DAYS:MAX_DAYS:WARN_DAYS:INACTIVE:EXPIRE_DATE:RESERVED
```

- **login**

Имя пользователя соответствует /etc/passwd.

- **hash**

Хеш пароля необратимое криптографическое преобразование.

- **LAST_DAY**

Дата последнего изменения пароля.

- **MIN_DAYS**



Через сколько дней можно будет поменять пароль.

- **MAX_DAYS**

Через сколько дней пароль устареет.

- **WARN_DAYS**

За сколько дней до того, как пароль устареет, напомнить о необходимости его смены.

- **INACTIVE**

Через сколько дней после того, как пароль устареет, заблокировать учётную запись пользователя.

- **EXPIRE_DATE**

Дата блокировки учётной записи.

- **RESERVED**

Зарезервированное поле, не используется.

Размещение базы данных паролей

БД паролей не должна быть доступна для чтения/записи обычным пользователям. Для изменения пароля предполагается использовать SUID/SGID утилиты, такие как `passwd`.

`/etc/shadow`

Традиционный для UNIX-подобных систем и ОС на основе ядра Linux вариант хранения паролей. В ОС «Альт» не используется по умолчанию, но поддерживается.

`/etc/tcb`

Используемый в ОС «Альт» формат базы хранения паролей, технически файл `shadow` разрезанный по строкам. Обеспечивает более строгие правила доступа к хэш-образам паролей.

```
# namei -om /etc/tcb/user1/shadow
f: /etc/tcb/user1/shadow
drwxr-xr-x root root /
```



```
drwxr-xr-x root root etc
drwx--x--- root shadow tcb
drwx--s--- user1 auth user1
-rw-r----- user1 auth shadow
```

Режимы доступа к элементам иерархии /etc/tcb

Инструменты работы с паролями пользователей

- **chage**

Смена временных ограничений УЗ/пароля.

```
# chage -m MIN_DAYS -M MAX_DAYS -I INACTIVE -d LAST_DAY -E EXPIRE_DATE -W
WARN_DAYS <user>
# chage -l <user>
```

Пример использования.

- **passwd**

Задаёт новый пароль пользователя. Суперпользователь меняет пароль другим, пользователь – себе.

Учётные записи групп

Файл **/etc/group**

```
group:x:GID:user1,user2,user3
```

Можно редактировать вручную, но лучше использовать соответствующие инструменты.

Управление группами

- **groupadd**

Добавление новой группы.

- **groupmod**

Изменение группы и добавление новых пользователей в группу.

- **groupdel**

Удаление группы.



Управление членством пользователя в группах

- **usermod**

Управление членством пользователя в группах.

- **gpasswd**

Управление составом группы.

Определение принадлежности пользователя к группам

- **id**

Информация о пользовательских идентификаторах процесса.

- **groups**

Список групп для текущего пользователя.

```
$ id -Gn [user1, user2 ...]  
$ groups [user1, user2 ...]  
$ grep <user> /etc/group
```

Информация про себя – без параметров. Про другого пользователя – указываем имя (не требует суперпользователя).

Информация по пользователям в системе

Пользователи, работающие в системе

- **who**

Пользователи работающие в системе.

```
$ who  
root          tty2          2024-01-22 10:00  
sysadmin      pts/0         2024-01-22 09:59 (:0.0)  
sysadmin      pts/1         2024-01-22 10:00 (example.com)
```

- **w**

Пользователи, работающие в системе.

```
$ w  
11:44:13 up 3:13, 1 user, load average: 0,95, 0,89, 0,57  
USER      TTY      LOGIN@  IDLE   JCPU   PCPU   WHAT
```



```
user1    tty1      08:31    3:13m   3:06    0.05s /bin/sh
/usr/lib/kf5/bin/startkde
```

Параметр	Описание
LOGIN@	Время входа в систему
IDLE	Время, с момента запуска какой-либо команды
JCPU	Время, использования CPU с входа в систему
PCPU	Время, использования CPU текущим процессом
WHAT	Процесс, который выполняется

История входов в систему

- **last**

История входов пользователей.

- файл **/var/log/wtmp**

*Утилита анализирует файл **/var/log/wtmp** – история логинов пользователей.*

```
$ last
user1 tty1 :0      Wed Oct 30 08:31  still logged in
root  pts/1 localhost Sun Oct 27 11:54 - 14:35  (02:40)
```

Механизмы получения административных полномочий в системе

Иерархия УЗ пользователей

- **UID = 0** – суперпользователь (root)

*Максимальные привилегии в системе (доступ не ограничивается).
Не рекомендуется производить интерактивный вход в систему под этой УЗ.*

- **0 < UID < UID_MIN****

Специальные/системные УЗ для запуска служб.

- **UID_MIN < UID < UID_MAX**

*Диапазон UID для пользовательских (операторских) УЗ.
useradd/newusers используют этот диапазон по умолчанию при создании пользовательских УЗ.*



```
# grep UID /etc/login.defs
# Min/max values for automatic UID selection in useradd.
UID_MIN          500
UID_MAX          60000
```

Использование операторских УЗ и получение административных полномочий

- Пользователь **не должен входить в систему** под учетной записью суперпользователя

Требования безопасности – не держать интерактивную сессию под суперпользователем, не запускать под суперпользователем пользовательские приложения, тем более сетевые.

Механизмы повышения полномочий

- явная передача полномочий – **su (switch user)**
- контролируемая передача полномочий – **sudo (switch user do)**
- запуск процессов со специальными атрибутами – **suid/sgid, capabilities**
- управляемое политиками повышение полномочий процессов – **PolicyKit**

Команда **su**

*Реализует **повторную регистрацию** пользователя в системе. УЗ для регистрации указывается в качестве параметра. Требуется знать (и ввести) пароль новой УЗ.*

```
$ su [-l|-L] [имя_пользователя] [-c command]
```

- опция **-l**

Запускает командный интерпретатор с параметрами окружения нового пользователя.

- опция **-c**

Запускает указанную команду, а не командный интерпретатор.

<https://altlinux.org/Su>



Команда **sudo**

- **Контролируемая** передача полномочий

Запускаются отдельные программы, а не сеанс командного интерпретатора. Перед передачей полномочия у пользователя может запрашиваться пароль своей УЗ.

```
$ head /etc/shadow
head: cannot open '/etc/shadow' for reading:
Permission denied
$ sudo head /etc/shadow
[sudo] password for sysadmin:
...
```

Получение через sudo доступа к файлу, недоступному для обычного пользователя.

```
# control sudowheel enabled
```

Разрешение использовать sudo членам группы wheel.

<https://altlinux.org/Sudo>

Синтаксис **/etc/sudoers**

пользователь хост = (д_пользователь:группа)опт: команды

- пользователь (или группа)

Кому можно выполнять

- хост

На каком узле

- д_пользователь:группа

В кого разрешено переходить

- команды

Что разрешено выполнять



Модуль 9 Конфигурация системы

Уровни конфигурации

Общесистемная конфигурация

```
/etc/
```

Настроечные файлы приложений и подсистем с параметрами, используемыми по умолчанию. Настраиваются системным администратором.

Пользовательская конфигурация (профиль)

```
/home/<user>/.*
```

Пользовательские параметры приложений. Переопределяют параметры, заданные общесистемно.

```
/etc/skel
```

Скелет пользовательского профиля. Копируется в домашний каталог пользователя при его создании и определяет, тем самым пользовательские параметры.

Рабочая конфигурация (контекст)

Конфигурация, задаваемая при запуске приложения за счет параметров командной строки или переменных пользовательского окружения. Переопределяет пользовательскую и системную конфигурацию.

Пользовательское окружение

Области действия переменных

- **Локальные** переменные

Определены только для текущего сеанса командного интерпретатора.



- **Переменные окружения**

Определены в текущем сеансе и во всех дочерних процессах. Могут таким образом использоваться для параметризации запускаемых приложений.

```
$ env
```

Отображение всех переменных окружения.

```
$ export VAR
```

Превращаем переменную с локальной областью видимости в переменную окружения.

Основные переменные окружения

- **PATH**

Пути поиска исполняемых файлов.

- **PS1**

Формат приглашения командной строки.

- **LANG**

Язык системы.

- **EDITOR**

Линейный редактор – для работы на терминалах с выводом на печатающее устройство с непрерывной подачей – такие как `ed` и `ex`.

- **VISUAL**

Используемый экранный редактор – для вывода на ЭЛТ-терминал и современные мониторы. В настоящий момент можно использовать любую из данных переменных.

```
$ LANG=C ls -l
```

Установка значения переменной для контекста запуска конкретного приложения.



Настройки командного интерпретатора

```
$ set
```

Установка переменных командного интерпретатора (настраивающих его работу).

```
$ help set
```

Справка по доступным для настройки параметрам.

Пользовательский профиль

Начальный и интерактивный командный интерпретатор

- **Начальный** командный интерпретатор (login shell)

Интерпретатор, выполняемый при входе пользователя в систему. Может быть запущен в интерактивном и неинтерактивном режиме.

- **Интерактивный** командный интерпретатор

Командный интерпретатор, запущенный с привязкой входных и выходных потоков ввода-вывода к терминалу.

Общесистемная конфигурация

- **Общесистемные настройки профиля (login shell)**

```
/etc/profile  
/etc/profile.d/*.sh
```

Системные настройки пользовательского профиля. Формирование окружения пользователя – переменные, значение `umask`. Ресурсные ограничения (`ulimit`).

- **Общесистемные настройки интерактивного командного интерпретатора**

```
/etc/bashrc  
/etc/bashrc.d/*.sh
```



*Системные настройки командного интерпретатора.
Корректировка переменных, специфичных для `bash`, переменных командного интерпретатора, установка `alias`-ов.*

```
/etc/bashrc.d/bash_completion.sh  
/etc/bash_completion.d/*
```

Настройки автозавершений в командной строке.

Пользовательская конфигурация

- Пользовательские настройки профиля (login shell)

```
~/.bash_profile  
~/.bash_login  
~/.profile
```

Пользовательский скрипт (используется один из трёх – `.bash_profile`, `.bash_login`, `.profile`, в ОС «Альт» по умолчанию – первый), выполняемый при запуске начального командного интерпретатора, после запуска `/etc/profile`.

- Пользовательские настройки интерактивного командного интерпретатора

```
/home/<user>/.bashrc
```

Файл выполняется при каждом интерактивном запуске командного интерпретатора. Вызывает общесистемный `/etc/bashrc`.

- Пользовательские настройки неинтерактивного командного интерпретатора (выполнение скриптов)

```
$ echo $BASH_ENV  
/home/<user>/.bashrc
```

При неинтерактивном запуске командного интерпретатора (выполнение скрипта) он выполняет файл, заданный этой переменной, если он существует. Т.е. по сути происходит тоже самое.



Модуль 10 Разграничение доступа

Базовая модель разграничения доступа

Дискреционный доступ

- Дискреционный доступ

Дискреционное (избирательное) управление доступом. Субъект является владельцем части объектов системы и для своих объектов решает, кто может с ними работать и какими методами доступа.

- Субъект

*Пользователь, а вернее процессы, работающие в его **контексте безопасности**. Субъекты объединяются в **группы**.*

- Объекты

*Ресурсы системы – файлы и т.п. Объекты объединяются в **контейнеры** (папки).*

- Право доступа

Субъект-метод-объект.

Принципы дискреционного доступа (ОС UNIX/Linux)

- владелец объекта

*Идентификатор владельца объекта и его приватной группы (UPG) хранится вместе с объектом – **атрибуты защиты файлов**.*

- установка **прав доступа** (umask)

*Устанавливаться по умолчанию по значению переменной **umask** или задаются при создании файла.*

- **привилегированный** пользователь

*root в UNIX-системах. В случае, если идентификатор пользователя равен **0 (root)**, система **не будет производить проверку** наличие права доступа у этого пользователя.*



Режимы доступа файловой системы

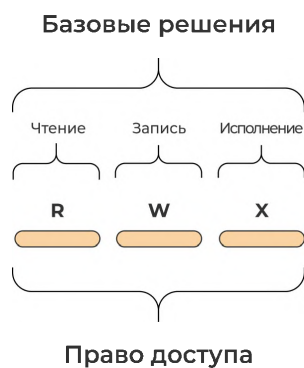


Рисунок 27:
Базовые
разрешения

- **r**

чтение

- **w**

запись

- **x**

выполнение

Режимы доступа для стандартных субъектов

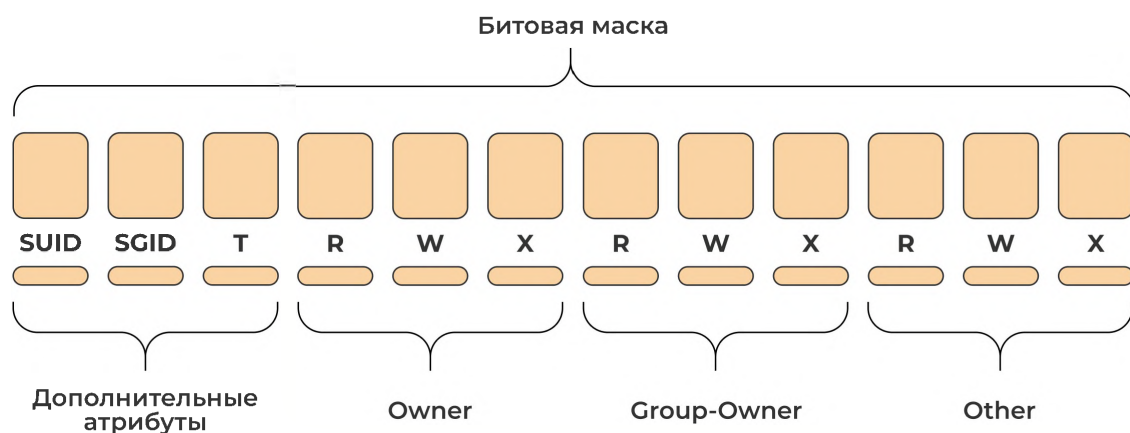


Рисунок 28: Биты доступа

- **owner**

Владелец файла

- **group-owner**

Группа-владелец файла

- **other**

Все остальные

Создание объектов ФС

При создании объекта его владельцем становится тот, кто его создал, группой-владельцем – приватная группа пользователя.

```
$ id
uid=1001(user1) gid=1001(user1) groups=1001(user1),4(adm)
$ touch file1
$ ls -l file1
-rw-r--r--. 1 user1 user1 0 Oct 21 10:18 file1
$ mkdir dir1
$ ls -l dir1
drwxr-xr-x. 2 user1 user1 0 Oct 21 10:19 dir1
```

Параметр **umask**

Разрешения при создании объектов ФС определяются наложением **umask** на базовые разрешения



- Базовые разрешения
 - для файлов: **rw-rw-rw-**
 - для каталогов: **rwxrwxrwx**

```
$ umask
0022
$ umask 027
$ touch sample
$ ls -l sample
-rw-r----- 1 user1 user1 0 Oct 28 20:14 sample
$ mkdir test-dir
$ ls -ld test-dir
drwxr-x--- 1 user1 user1 4096 Oct 28 20:25 test-dir
```

Влияние `umask` на режимы доступа создаваемых файлов.

```
# grep -i umask /etc/login.defs
UMASK 076
# grep -i umask /etc/profile
umask 022
```

*Устанавливается по умолчанию в **/etc/login.defs**. Дополнительно может задаваться в стартовых скриптах пользовательской сессии.*

Изменение атрибутов безопасности файлов

- Атрибуты безопасности файлов

Режимы доступа, владелец, группа-владелец.

Управление владением файлов/каталогов

- утилита **chown**

Пользователя-владельца может менять только суперпользователь.

```
# chown user1 file1
# chown user1:group1 file2
# chown -R user2 directory1
```

*Установка владельца файла – **chown**.*

```
# chown jane /tmp/file1
# ls -l /tmp/file1
-rw-rw-r-- 1 jane sysadmin 0 Dec 19 18:44 /tmp/file1
# chown jane:users /tmp/file2
```



```
# ls -l /tmp/file2  
-rw-r--r-- 1 jane users 0 Dec 19 18:53 /tmp/file2
```

- утилита **chgrp**

Изменение группы-владельца. Пользователи у своих файлов могут менять группу-владельца на одну из тех групп, к которым они принадлежат.

```
$ chgrp group1 file1  
$ chgrp -R group1 directory2
```

*Установка группы владельца – **chgrp(chown)**.*

Управление режимами доступа

Пользователь может менять режимы доступа для своих файлов.

```
$ chmod [-R] <perm> <file>
```

Установка разрешения на файлы/каталоги.

- **perm**

Режим доступа

- **u+x**

Исполнение для владельца

- **g-w**

Убрать запись для группы

- **664**

Установка в восьмеричной форме



Действие режимов доступа на файлы и каталоги

ОСТ	BIN	Mask	Права на файл	Права на каталог
0	000	- - -	отсутствие прав	отсутствие прав
1	001	- - x	права на выполнение	доступ к файлам и их атрибутам
2	010	- w -	права на запись	отсутствие прав
3	011	- w x	права на запись и выполнение	все, кроме доступа к именам файлов
4	100	r - -	права на чтение	только чтение имен файлов
5	101	r - x	права на чтение и выполнение	чтение имен файлов и доступ файлам и их атрибутам
6	110	r w -	права на чтение и запись	только чтение имен файлов
7	111	r w x	полные права	все права

Рисунок 29: Действие битов доступа

Особенности действия разрешений

- Действие разрешений с корня

Для успешного доступа пользователя к файлу необходимо, чтобы его «пропустили» все каталоги начиная с /.

- Работа разрешений для владельца

Если доступ производит владелец файла, проверяются только разрешения владельца.

- Работа разрешений для членов группы-владельца

Если доступ производит пользователь из группы-владельца, но не владелец – проверяются только разрешения группы.

Дополнительные атрибуты для исполняемых файлов

Пользовательские атрибуты процесса

- **RUID**

Реальный идентификатор пользователя – кто запустил.

- **EUID**



Эффективный идентификатор пользователя.

- **RGID**

Реальный идентификатор группы (первичная группа пользователя).

- **EGID**

Эффективный идентификатор группы (первичная группа пользователя).

- **GROUPS**

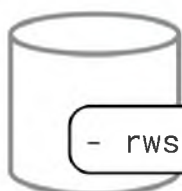
Все группы, в которые входит EUID.

RUID=EUID, RGID=EGID

Для большинства процессов, т.е. кто запустил, от того и разрешения доступа. Исключения составляют SUID/SGID-программы.

Атрибуты процесса при запуске прикладного ПО

uid=500(user) gid=500(user) группы=500(user),10(wheel)



- rwsr-xr-x 1 root root /bin/whoami



LOAD

Process
CMD=/bin/whoami
RUID=500
EUID=500
RGID=500
EGID=500
GROUPS=500, 10

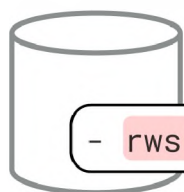
Рисунок 30: Запуск прикладного ПО

Прикладное ПО, как правило, работает в контексте безопасности запустившего его пользователей. При этом доступ к ресурсам разрешается или запрещается в зависимости от пользовательских атрибутов процесса.



SUID/SGID процессы

uid=500(user) gid=500(user) группы=500(user),10(wheel)



- rwsr-xr-x 1 root root /bin/passwd



LOAD

Process
CMD=/bin/passwd
RUID=500
EUID=0
RGID=500
EGID=500

Рисунок 31: SUID/SGID-процессы

Специальные биты защиты, согласно которым процесс получает права не того пользователя, который процесс запустил, а владельца исполняемого файла.

```
rwsr-xr-x root root  
rwxr-sr-x root root
```

- SUID-приложение

```
# ls -l /bin/su  
-rws--x--- 1 root wheel 31072 июл 3 2020 /bin/su
```

Разрешение выполнения у группы wheel, при выполнении получает EUID root.

- SGID-приложение

```
# ls -l /usr/bin/wall  
-rwx--s--x 1 root tty 18624 ноя 12 2019 /usr/bin/wall
```

Разрешение выполнения у всех, при выполнении EGID tty.

Поиск SUID/SGID-файлов в системе

```
$ find / -type f -perm /4000  
$ find / -type f -perm /2000
```

SUID/SGID-программы являются критическими с точки зрения безопасности системы объектами – Их надо знать – За ними надо следить.



Дополнительные атрибуты для каталогов

SGID на каталог для совместной работы

- **T3:**
 - Несколько разных пользователей с разными первичными группами должны работать над одним проектом.
 - Данные проекта должны быть доступны им и больше никому.
 - Пользователи могут управлять доступом членов группы **team** к своим файлам в этом каталоге.

```
$ groupadd team
$ usermod -aG team bob
$ usermod -aG team sue
$ usermod -aG team tim
$ mkdir /srv/project
$ chgrp team /srv/project
$ chmod 770 /srv/project
$ ls -ld /srv/project
drwxrwx--- 2 root team 4096 Oct 30 14:11 /srv/project
```

Назначение режимов доступа.

```
$ ls -ld /srv/project
drwxrwx--- 2 root team 4096 Oct 30 14:11 /srv/project
```

*Создание пользователем **bob** файла в каталоге **/srv/project**:*

```
$ touch file.txt
$ ls -l
-rw-r----- 1 bob bob 100 Oct 30 23:21 file.txt
```

*Другие пользователи из группы **team** не смогут работать с этим файлом.*

- T3 failed!

```
$ chmod g+s /srv/project
$ ls -ld /srv/project
drwxrws--- 2 root team 4096 Oct 30 14:11 /srv/project
```

*Создание пользователем **bob** файла в каталоге **/srv/project**:*

```
$ touch file.txt
$ ls -l
-rw-r----- 1 bob team 100 Oct 30 23:21 file.txt
```



Другие пользователи из группы **team** могут читать этот файл, т.е. получаем возможность совместной работы.

Sticky bit на каталог для совместной работы

- Действие **sticky bit** на каталог

Используется для запрета пользователям удалять из общих каталогов чужие файлы.

- Каталог **/tmp**

```
$ ls -ld /tmp
drwxrwxrwt 1 root root 4096 Oct 30 14:17 /tmp
```

Без **sticky bit** пользователь с правами записи на каталог может как создавать файлы в каталоге, так и удалять уже существующие файлы, независимо от разрешений на них.

```
$ chmod o+t <directory>
$ chmod 1775 <directory>
$ chmod o-t <directory>
$ chmod 0775 <directory>
```

С установленным на каталог **sticky bit** файлы, в нём, могут быть удалены только их владельцем или суперпользователем.



Модуль 11 Управление процессами

Атрибуты процессов

Основные атрибуты

Атрибут	Описание
PID	Идентификатор процесса
PPID	Идентификатор родительского процесса
EUID	Эффективный идентификатор пользователя
RUID	Реальный идентификатор пользователя
EGID	Эффективный идентификатор группы
RGID	Реальный идентификатор группы
GROUPS	Перечень групп, в которые входит EUID
SID	Идентификатор сеанса (session ID)
PGID	Идентификатор группы процессов
TPGID	Идентификатор терминальной группы
TTY	Терминал, к которому привязан процесс
NICE	Число – возможность отдавать CPU другим

Идентификаторы процессов

Каждый процесс получает уникальный идентификатор (PID) со значениями от 2 (1 – init) до значения PID_MAX-1. Текущее значение PID_MAX можно посмотреть командой:

```
# cat /proc/sys/kernel/pid_max
```

- **/proc/sys/kernel/pid_max**

При достижении максимального значения (система работает долго), начинают задействоваться свободные идентификаторы с начала.

```
$ ls /proc/<pid>
```

Иерархия процессов в системе

- **init**



Ядро после загрузки запускает процесс **init** с идентификатором (PID) 1. Этот «первый» процесс является прародителем всех остальных процессов.

- **systemd**

В современных дистрибутивах роль **init** выполняет **systemd**.

- **pstree**

Иерархию процессов можно посмотреть при помощи **pstree**.

```
$ pstree
init--+-cron
      |-login---bash---pstree
      |-named---18*[{named}]
      |-rsyslogd---2*[{rsyslogd}]
      `--sshd
```

Процессы, группы и сеансы

Для удобства управления процессами при помощи сигналов процессы объединяются в **группы и сеансы**

- **Сеанс (session)**

связывает процессы с **управляющим терминалом**, когда пользователь входит в систему, все создаваемые им процессы будут принадлежать сеансу, связанному с его текущим терминалом. Процесс, создавший сеанс, имеет идентификатор PID, совпадающий с идентификатором SID, называется лидером сеанса.

- **Группы**

процессов внутри сеансов позволяют управлять, какие из процессов сеанса работают **на переднем фоне**. Процесс, создавший группу, имеет идентификатор PID, совпадающий с идентификатором PGID, называется лидером группы. Только одна группа сеанса называется «терминальной» TGPID, является группой переднего фона (foreground).



Инструменты анализа процессов в системе

Утилита **ps**

Выводит список процессов

- **ps**

Вывод процессов текущего сеанса командного интерпретатора (привязанных к текущей TTY).

- **TIME**

Процессорное время, потребленное процессом.

- **ps x**

Вывод всех процессов текущего пользователя, независимо от TTY.

- **STAT**

Текущее состояние процесса.

- **ps aux**

*Вывод всех процессов – **a** в удобном – **u** формате.*

- **ps f**

Вывод процессов в древовидном формате.

Опции **ps**

- 3 варианта задания опций

BSD, UNIX, многосимвольные опции

```
$ ps aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
...
```

BSD-style

```
$ ps -ef
UID      PID      PPID  C STIME TTY          TIME CMD
...
```



UNIX-стайл

- **-o**

Указание выводимых полей.

```
$ ps -o pid,user,pri,stat,cmd
```

Поля вывода **ps**

Поле	Описание
PID/PPID	Идентификатор процесса/родительского процесса
UID/USER	Идентификатор/имя пользователя
CMD	Командная строка процесса
TTY	Терминал, к которому привязан процесс
STAT	Состояние процесса
%CPU(C)	Используемые ресурсы процессора
%MEM	Процент использования физической памяти
VSZ(VIRT)	Виртуальный образ, все что доступно процессу
RSS(RES)	Резидентная память процесса, без выгруженной
STIME	Время запуска процесса
TIME	Потребленное процессорное время

Другие инструменты

- **top**

Интерактивная (TUI-утилита).

- **htop**

Альтернатива. Одноименный пакет.

- **btop**

Альтернатива. Одноименный пакет.

- GUI-инструменты

Менеджер процессов/задач и т.п. используемой графической среды.



Состояния процессов

Таблица состояний

STAT	Описание
R	Выполняется
S	Готов – ожидает выполнения (спит)
D	Ожидает – приостановлен (ввод-вывод)
T	Остановлен (принудительно)
I	Поток ядра
Z	Зомби – завершился, но не был удален
<	Высокоприоритетный процесс
N	Низкоприоритетный процесс
s	Лидер сеанса
L	Часть страниц блокирована в памяти
I	Мультипоточный (нити)
+	Работает в группе переднего плана



Жизненный цикл процесса

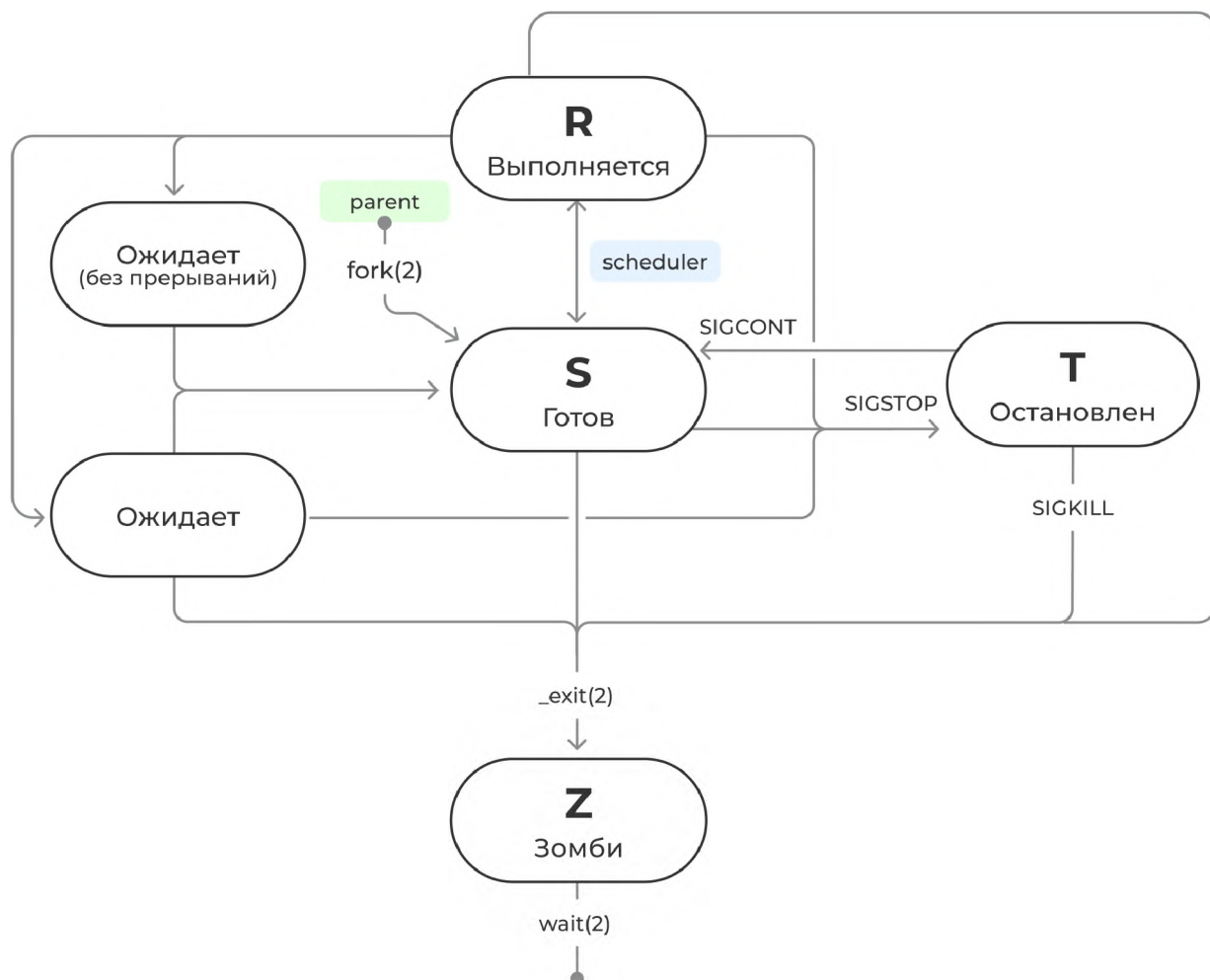


Рисунок 32: Жизненный цикл процесса

Сигналы

Средство для изменения состояния процессов из-вне

- Обработываемые самим **процессом** (игнорируемые)

Могут передаваться целевому процессу (и тогда могут быть проигнорированы им).

- Обработываемые **ядром**



Или могут обрабатываться ядром и таким образом срабатывают всегда.

Посылка сигналов

```
$ kill [ -SIGNAL ] PID
$ killall [ -SIGNAL ] { user | name }
$ pkill [options] pattern
```

Часто используемые сигналы

№	Имя	Описание
1	SIGHUP	Обрыв связи, переинициализация
2	SIGINT	Прервать (Ctrl+C)
9	SIGKILL	Уничтожить (не игнор)
15	SIGTERM	Завершить (сигнал по умолчанию)
18	SIGCONT	Продолжить после STOP
19	SIGSTOP	Приостановить (не игнор)
20	SIGTSTP	Стоп с клавиатуры (Ctrl+Z)

```
$ kill -9 87564
```

Работа с заданиями (jobs)

Задания

Функция командного интерпретатора.

```
$ dd if=/dev/zero of=/dev/null &
[1] 6453
```

Запуск процесса в фоновом режиме (создание задания).

```
$ jobs
[1]-  Запущен      dd if=/dev/zero of=/dev/null &
[2]+  Запущен      dd if=/dev/zero of=/dev/null &
```

Просмотр списка заданий.

Управление состоянием заданий

```
$ fg %1
```



Возврат процесса на передний план.

- **Ctrl+Z** – сигнала TSTP

```
$ dd if=/dev/zero of=/dev/null  
^Z  
[2]+  Остановлен    dd if=/dev/zero of=/dev/null  
$
```

Приостановка процесса переднего плана (без его завершения).

```
$ bg %3  
[3]+ dd if=/dev/zero of=/dev/null &
```

Перевод приостановленного процесса в работу в фоновом режиме.

Приоритеты выполнения процессов

Атрибут процесса NICE

- **Низкоприоритетный процесс** (STAT=N)

Получает процессорное время только после того, как будут обслужены процессы с более высоким приоритетом.

- **Niceness** (уступчивость)

*Свойство процесса оставлять процессорное время другим процессам – процессы с **высоким приоритетом** называют менее уступчивыми – **меньше nice**; с **низким приоритетом** – более уступчивые – **больше nice**.*

```
-20 <= NICE <= 19, default=0
```

Диапазон значений NICE.

Утилиты **nice** и **renice**

*Пользователь может только увеличивать значение **nice**, для уменьшения необходимы привилегии суперпользователя.*

- **nice**

```
$ nice -n 10 low-prg
```



```
# nice -n -20 high-prg
```

Запуск процесса с определенным значением **nice**.

- **renice**

Изменение значения **nice** для работающего процесса.

```
$ renice +5 17489
17489 (process ID) старый приоритет 0, новый приоритет 5
$ renice +2 $(pgrep firefox)
2990 (process ID) old priority 0, new priority 2
2993 (process ID) old priority 0, new priority 2
```

- Связь значения nice и приоритета

Чем больше **nice**, тем меньше приоритет и наоборот.



Модуль 12 Удаленный доступ

Сетевые настройки

Сетевые адаптеры

- Сетевые адаптеры

Подключаются к шинам PCI/PCIe/USB.

- Драйвера (модули) адаптеров

Ядро загружает соответствующие подключенным адаптерам модули (драйвера)

```
$ lspci -k | grep -A2 Ethernet
04:00.0 Ethernet controller: Realtek Semiconductor Co., Ltd.
RTL8111/8168/8411 PCI Express Gigabit Ethernet Controller (rev 10)
Subsystem: Lenovo Device 3884
Kernel driver in use: r8169
```

Сетевые интерфейсы

- Сетевые интерфейсы

Модули создают представление сетевых адаптеров в системе в виде

```
$ ls /sys/class/net
```

Список сетевых интерфейсов

```
$ ls /sys/class/net
docker0 eth0 lo wlan0
```

Утилита **ip**

```
$ ip [ OPTIONS ] OBJECT { COMMAND | help }
```

Управление настройками стека IP на сетевых интерфейсах.

```
$ ip link show
$ ip l
$ ip l show eth0
```



Сводка информации по интерфейсам/конкретному интерфейсу.

```
$ ip -s l show eth0
```

Информация по конкретному интерфейсу, включая статистику.

```
$ ip addr show  
$ ip a  
$ ip a show dev eth0  
$ ip -br a
```

Просмотр IP-адресов по интерфейсу.

```
$ ip link set ppp0 up  
$ ip link set ppp0 down
```

Включить/выключить интерфейс.

- `ip(8)`

Настройка сетевых интерфейсов средствами `ip`

```
$ ip a add 192.168.1.12 dev eth0
```

Назначение IP-адреса.

```
$ ip a add 192.168.1.12/24 dev eth0
```

Назначение адреса с указанием маски.

```
$ ip route add 0.0.0.0/0 via 10.0.0.1
```

- `ip-address(8)`
- `ip-route(8)`

Сетевые подсистемы ОС «Альт»

Etcnet

Собственная система настройки сети в ОС «Альт». Настройка через редактирование конфигурационных файлов или средствами ЦУС.



```
/etc/net
```

Каталог с настройками по интерфейсам.

```
# ifdown eth0
```

Отключить интерфейс.

```
# ifup eth0
```

Включить интерфейс.

NetworkManager

Система управления сетевыми установками, имеющая собственные утилиты управления, включая графические, интегрированные с графическим пользовательским окружением. Значительно упрощает настройку сети в современном динамичном сетевом окружении

- **GUI** NetworkManager

GUI-утилиты в каждом графическом окружении Linux вызываются через иконки доступа к сети на панелях и т.п.

- **nmtui**

Интерактивная текстовая утилита, позволяющая выполнять сетевые настройки (ncurses).

- **nmcli**

Интерфейс командной строки, наиболее низкоуровневый, для использования в скриптах и т. п.

Systemd-networkd

Системная служба для управления сетевыми настройками. Её задачей является обнаружение и настройка сетевых устройств по мере их появления, а также создание виртуальных сетевых устройств.

```
/etc/systemd/network/
```



Каталог с настройками по интерфейсам.

```
# networkctl down eth0
```

Отключить интерфейс.

```
# networkctl up eth0
```

Включить интерфейс.

Режимы настройки сетевых интерфейсов

- **Etcnet**

Изменение сетевых параметров и состояния сетевых интерфейсов доступно только для суперпользователя через редактирование файлов в `/etc/net` или через ЦУС.

- **NetworkManager(etcdnet)**

Изменение сетевых параметров доступно только для суперпользователя через редактирование файлов в `/etc/net` или через ЦУС. Изменение состояния сетевых интерфейсов доступно обычным пользователям через компоненты графической среды.

- **NetworkManager(native)**

Изменение сетевых параметров и состояния сетевых интерфейсов доступно обычным пользователям через компоненты графической среды.

- **systemd-networkd**

Изменение сетевых параметров и состояния сетевых интерфейсов доступно только для суперпользователя через редактирование файлов в `/etc/systemd/network` или через ЦУС.

Режим	Параметры сети	Состояние интерфейса
Etcnet	/etc/net, ЦУС	ifup/ifdown
NetworkManager(etcdnet)	/etc/net, ЦУС	NetworkManager
NetworkManager(native)	NetworkManager	NetworkManager
systemd-networkd	/etc/systemd/network, ЦУС	networkctl down/networkctl up



Разрешение имен

Методы разрешения имен

```
/etc/nsswitch.conf
```

Настройка коммутатора системы имен.

Формат файла **/etc/hosts**

```
cat /etc/hosts
```

Статическое разрешение.

```
<IP> <name1> <name2>
```

Формат строки файла /etc/hosts.

Формат файла **/etc/resolv.conf**

```
/etc/resolv.conf
```

Конфигурационный файл DNS-резолвера (клиента). Содержит указание на DNS-сервер. Не предназначен для непосредственного редактирования. Получает содержимое на основании параметров разрешения имен сетевых интерфейсов.

```
nameserver <IP>
```

Указание сервера имен

- **resolvconf**

Утилита, формирующая общесистемную конфигурацию DNS-резолвера при активации интерфейса.

Утилиты сетевой диагностики

ping/ping6 – доступность IP-узла

```
$ ping [OPTIONS] [IP|NAME]  
$ ping6 [OPTIONS] [IP|NAME]
```



Отправляет диагностические ICMP-сообщения на удаленный узел.

- **-с X**

Количество сообщений (по умолчанию – бесконечно).

```
$ ping -c 4 192.168.1.1
```

Проверка связности с удаленным узлом.

ss – просмотр сетевых соединений

```
$ ss
```

Просмотр информации о сетевых и файловых сокетах

- **-a**

Все активные сокеты

- **-l**

Сокеты в состоянии LISTEN

- **-t/ -u**

Сокеты TCP/сокеты UDP

- **-n**

Числовое отображение узлов и портов

- **-p**

Отображение процессов (требуется root)

- **-s**

Статистика

```
$ ss -s  
$ ss -atnp
```

Утилиты dig/host/nslookup

```
$ dig example.com  
$ host example.com
```



Тестирование процесса разрешения имен, отправка запросов на разрешение имен, отправка определенных запросов к конкретному серверу.

SSH – Secure Shell



Рисунок 33: Схема работы SSH

Протокол для безопасного удалённого входа в систему и предоставления других безопасных служб поверх небезопасной сети. Обеспечивает аутентификацию сервера, стойкое шифрование передаваемых данных, контроль целостности. > 22 порт, клиент-серверный

Подключение к SSH-серверу

```
$ ssh user@host -p port
```

порт по умолчанию 22

```
$ scp path/file user@host:/dir/  
$ scp user@host:/path/file dir/
```

копирование файлов

Настройка SSH (OpenSSH)

- OpenSSH



Модуль 12

131

Удаленный доступ

Наиболее распространенная реализация – **OpenSSH** (пакеты *openssh, openssh-server*)

```
/etc/openssh/sshd_config
```

Конфигурация сервиса *sshd*.

```
/etc/openssh/ssh_config  
$HOME/.ssh/config
```

Конфигурация клиента *ssh*.

Ключи хоста SSH на сервере

```
/etc/openssh/ssh_host_<alg>_key  
/etc/openssh/ssh_host_<alg>_key.pub
```

пары *приватный* (доступен на чтение только *root*) и *публичный* (доступен всем).

Ключи хоста SSH на клиенте

```
/etc/openssh/known_hosts
```

ключи известных серверов (в системной настройке клиента).

```
$ cat $HOME/.ssh/known_hosts  
e530 ssh-ed25519 AAAAC3NzaC1lZDI. . .1Swog0IIwmRxvGb  
192.168.1.37 ssh-ed25519 AAAAC3NzaC . . . 7oLeE5sIb6
```

в домашних каталогах пользователей.

Настройка сервера SSH

```
/etc/openssh/sshd_config
```

Конфигурация сервиса *sshd*.

```
PermitRootLogin  
PasswordAuthentication  
AllowUsers user1 user2  
AllowGroup group1 group2
```




Модуль 12

132

Удаленный доступ

Основные параметры настройки.

```
$ systemctl restart sshd
```

Перезапуск службы.

Аутентификация по пользовательским ключам

Для использования аутентификации по паре открытый-закрытый ключ.

```
$ ssh-keygen [options]
```

Генерация пары ключей на клиенте.

```
$ ssh-copy-id user@192.168.56.102
```

Копирование пользовательских ключей на сервер.

```
~/.ssh/authorized_keys
```

Расположение пользовательских ключей на сервере. В домашнем каталоге того пользователя, под которым мы работаем на сервере.



Модуль 13 Приложение. Сводка по командам

1. Навигация по дереву каталогов

1.1. Команда **ls**

- Вывод информации по файлам и каталогам

```
$ ls [options] [dir|file]
```

- Опции **ls**
 - **-lh** – длинный вывод с человеческими единицами
 - **-a** – включая скрытые файлы
 - **-d** – информация по каталогу, а не по содержимому
 - **-tr** – сортировка по времени в обратной последовательности
 - **-S** – сортировка по размеру
- *ls(8)*

1.2. Команда **cd**

- Смена текущего каталога

```
$ cd [dir]
```

- Специальные символы
 - **-** – ранее используемый каталог
 - **..** – на уровень вверх
 - **~** – домашний каталог текущего пользователя
- *cd --help*

2. Просмотр текста

2.1. Утилита **cat**

- Конкатенация файлов (зачастую используется для вывода на экран)

```
$ cat [ПАРАМЕТР] [ФАЙЛ]...
```

- Опции **cat**
 - **-n** – нумерует все строки выходного файла, начиная с 1
- *cat(8)*

2.2. Утилита **head**

- Вывод начала файла



```
$ head [ОПЦИЯ]... [ФАЙЛ]...
```

- Опции **head**
 - **-n N** – вывод первых N строк каждого файла
 - **-c N** – вывод первых N байт каждого файла
- *head(8)*

2.3. Утилита tail

- Вывод конца файла

```
$ tail [опция]... [ФАЙЛ]...
```

- Опции **tail**
 - **-n N** – вывод последних N строк каждого файла
 - **-c N** – вывод последних N байт каждого файла
 - **-f** – интерактивно выводить поступающие данные
- *tail(8)*

2.4. Утилита grep - General Regular Expression Print

- Поиск подстроки (регулярного выражения) в тексте

```
$ grep [OPTION...] PATTERNS [FILE...]
```

- Опции **grep**
 - **--color** – подсвечивать вывод
 - **-c** – количество найденных вхождений
 - **-n** – отображать порядковый номер строчек
 - **-v** – инвертировать поиск – строки, где не встретился шаблон
 - **-i** – игнорировать регистр символов
 - **-w** – искать только вхождение слов, целиком соответствующих шаблону
- *grep(8)*

2.5 Утилита cut

- Вырезание текста по полям

```
$ cut [опции]... [ФАЙЛ]...
```

- Опции **cut**
 - **-d** – разделитель, по умолчанию Tab
 - **-f** – указание номеров столбцов
 - **-c** – указание порядковых номеров символов



- `cut(8)`

2.6. Утилита **wc**

- Статистика текста (Word Count) – количество строк, слов, байт

```
$ wc [опция]... [ФАЙЛ]...
```

- Опции **wc**
 - **-l** – только количество строк
 - **-w** – только слов
 - **-c** – только байт
 - **-m** – количество символов
- `wc(8)`

2.7. Утилита **nl**

- Нумерация строк

```
$ nl [опция]... [ФАЙЛ]...
```

- `nl(8)`

3. Организация хранения данных

3.1. Утилита **df**

- Отображает использование подключенных файловых систем

```
$ df [OPTIONS]
```

- Опции **df**
 - **-h** – human-воспринимаемые единицы (в alias по умолчанию)
 - **-T** – отображать тип файловой системы
 - **-i** – индексные дескрипторы
- `df(8)`

4. Управление файлами

4.1. Утилита **touch**

- Смена даты изменения существующего файла, создание файла при его отсутствии

```
$ touch [file]
```

- `touch(8)`



4.2. Утилита **cp**

- Копирование файлов и каталогов

```
$ cp [OPTIONS] [src] [dest]
```

- Опции **cp**
 - **-v** – отображение статуса
 - **-i** – интерактивный режим - запросит перед перезаписью существующего файла
 - **-n** – копировать без перезаписи
 - **-r** – копировать каталоги рекурсивно
- *cp(8)*

4.3. Команда **mv**

- Перемещение/переименование файлов/каталогов

```
$ mv [src] [dest]
```

- Опции **mv**
 - **-v** – отображение статуса
 - **-i** – интерактивный режим – запросит перед перезаписью существующего файла
 - **-n** – перемещать без перезаписи
- *mv(8)*

4.4. Команда **rm**

- Удаление файлов и каталогов

```
$ rm [OPTIONS] [file|dir]
```

- Опции **rm**
 - **-i** – интерактивный режим – запросит перед удалением существующего файла
 - **-r** – удалять каталоги рекурсивно
 - **-f** – удалять без вопросов
- *rm(8)*

4.5. Команда **mkdir**

- Создание каталога

```
$ mkdir [dir]
```

- *mkdir(8)*



4.6. Команда **rmdir**

- Удаление пустого каталога

```
$ rmdir [dir]
```

- *rmdir(8)*

4.7. Команда **du**

- Место на диске, занимаемое содержимым каталогов

```
$ du [ПАРАМЕТР]... [ФАЙЛ]...
```

- Опции **du**
 - **-c** – выводить общий итог
 - **-s** – выводить только суммарный итог
 - **-d N** – отображать данные до вложенности N
 - **-h** – в человеческих единицах
- *du(8)*

4.8. Другие утилиты

- Поблочное копирование

```
$ dd
```

- Создание файлов заданного размера/изменение размеров существующих

```
$ truncate
```

- Создание жестких и символических ссылок

```
$ ln
```

- Содержимое файлового дескриптора

```
$ stat
```

- Информация о содержимом файла

```
$ file
```

5. Архивирование и сжатие данных

- Объединение файлов в большой (создание архива)

```
$ tar
```

- Сжатие алгоритмом Лемпеля-Зива (LZ77)



```
$ gzip
```

- Сжатие алгоритмом Burrows-Wheeler

```
$ bzip2
```

- Сжатие алгоритмом LZMA

```
$ xz
```

6. Просмотр информации об оборудовании

- Просмотр информации о блочных устройствах

```
$ lsblk
```

- Просмотр атрибутов блочных устройств

```
$ blkid
```

- Информация о процессоре

```
$ lscpu
```

- Список устройств на шине USB

```
$ lsusb
```

- Список устройств на шине PCI/PCIe

```
$ lspci
```

- Информация об использовании памяти

```
$ free
```

7. Утилиты для работы с дисковыми разделами

- Стандартная консольная интерактивный утилита для управления дисковыми разделами

```
# fdisk <disk>
```

- Меню в текстовом (TUI) интерфейсе

```
# cfdisk <disk>
```

- Утилита для работы с дисковыми разделами из проекта GNU. работает как в интерактивном, так и в командном режимах. Позволяет менять размеры разделов



```
# parted <disk>
```

- GUI-версия parted

```
# gparted
```

8. Работа с файловыми системами

- Создание файловой системы

```
# mkfs
```

- Монтирование файловой системы

```
# mount
```

- Отмонтирование файловой системы

```
# umount
```

9. Настройка разграничения доступа

- Изменение владельца файла

```
# chown
```

- Изменение группы-владельца

```
$ chgrp
```

- Изменение режимов доступа

```
$ chmod
```

- Получение расширенных списков доступа (ACL) файла

```
$ getfacl
```

- Установка расширенных списков доступа (ACL) файла

```
$ setfacl
```

- Просмотр расширенных атрибутов файла

```
$ lsattr
```

- Установка расширенных атрибутов файла

```
$ chattr
```




10. Управление пользователями, паролями, группами

- Создание УЗ пользователей в пакетном режиме

```
# newusers
```

- Удаление существующей учётной записи.

```
# userdel
```

- Смена временных ограничений УЗ/пароля (пакет shadow-change)

```
$ chage
```

- Задаёт новый пароль пользователя. Суперпользователь может менять пароль другим, пользователь – только себе.

```
$ passwd
```

- Добавление новой группы

```
# groupadd
```

- Изменение группы и добавление новых пользователей в группу

```
# groupmod
```

- Удаление группы

```
# groupdel
```

- Управление составом группы

```
# gpasswd
```

10.1. Утилита useradd

- Добавление учётной записи пользователя

```
# useradd [OPTIONS] USER
```

- Опции **useradd**
 - **-m** – создавать домашний каталог (CREATE_HOME)
 - **-b** – базовый каталог для домашних каталогов (HOME)
 - **-d** – указание пути домашнего каталога
 - **-s** – указание командного интерпретатора (SHELL)
 - **-k** – шаблон профиля пользователя – что попадет в домашний каталог (SKEL)
 - **-c** – поле GECOS – описание/комментарий



- **-g** – указание первичной группы пользователя – UPG
- **-u** – указание UID
- **-G** – членство в дополнительных группах

`-useradd(1)`

10.2. Утилита **usermod**

- Изменение данных учётной записи.

```
# usermod OPTIONS USER
```

- Опции **usermod**
 - **-d** – указание пути домашнего каталога
 - **-s** – указание командного интерпретатора
 - **-k** – шаблон профиля пользователя - что попадет в домашний каталог (SKEL)
 - **-c** – поле GECOS – описание/комментарий
 - **-g** – указание первичной группы пользователя – UPG
 - **-u** – указание UID
 - **-G** – членство в дополнительных группах
 - **-L** – блокировка УЗ
 - **-U** – разблокировка

`-usermod(1)`

11. Информация о пользователях

- Информация о пользовательских идентификаторах процесса

```
$ id
```

- Список групп для текущего(указанного) пользователя

```
$ groups
```

- Пользователи работающие в системе

```
$ who
```

- Пользователи, работающие в системе

```
$ w
```

- История входов пользователей

```
$ last
```



12. Работа с процессами

- Список работающих процессов с их атрибутами

```
$ ps
```

- Дерево процессов

```
$ pstree
```

- Посылка сигнала процессу по идентификатору

```
$ kill [-SIG] PID
```

- Посылка сигнала процессу по имени/пользователю

```
$ killall [-SIG] {user|name}
```

- Поиск процессов по заданным критериям

```
$ pgrep [OPT]
```

- Посылка сигналов процессам, отобранным по заданным критериям

```
$ pkill [OPT]
```

- Запуск процесса с заданным значением NICE

```
$ nice [NUM] prog
```

- Изменение значения NICE для работающего процесса

```
$ renice [NUM] pid
```

13. Сетевые утилиты

- Диагностика/изменение параметров сетевого интерфейса

```
$ ip
```

- Статистика по сетевым/файловым сокетам

```
$ ss
```

- Проверка связности с узлом средствами ICMP

```
$ ping
```

- Включить сетевой интерфейс (etctnet)

```
# ifup <int>
```

- Выключить сетевой интерфейс (etctnet)



```
# ifdown <int>
```

- Управление сетевыми интерфейсами и соединениями (NetworkManager)

```
$ nmcli
```

- Управление сетевыми интерфейсами и соединениями (systemd-networkd)

```
# networkctl
```